Google File System

Slides by Raed Almomani

Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- GFS design
- Read
- Write
  - Consistency
- Append
- Fault tolerance
- Summary

Review: Distributed systems

- Distributed system: Multiple computers cooperating on a task
- MapReduce: Distributed system for compute
  - Run a program that would be too slow on one computer
  - Last time
- Google File System (GFS): Distributed system for storage
  - Store more data than fits on one computer
  - Today

Big storage examples

- When do we need storage that's too big for one computer?

- Store server logs from thousands ~ millions of servers
- Store a copy of the entire web
  - Needed to build a search engine inverted index ("look up table")
  - Several PB of HTML files
- Archive social media data
  - For millions ~ billions of users

Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- **GFS design**
- Read
- Write
  - Consistency
- Append
- Fault tolerance
- Summary

Google File System (GFS)

- Google File System: 2003

- Problem:
  - Store enormous amount of data
  - Don't lose data when a server fails

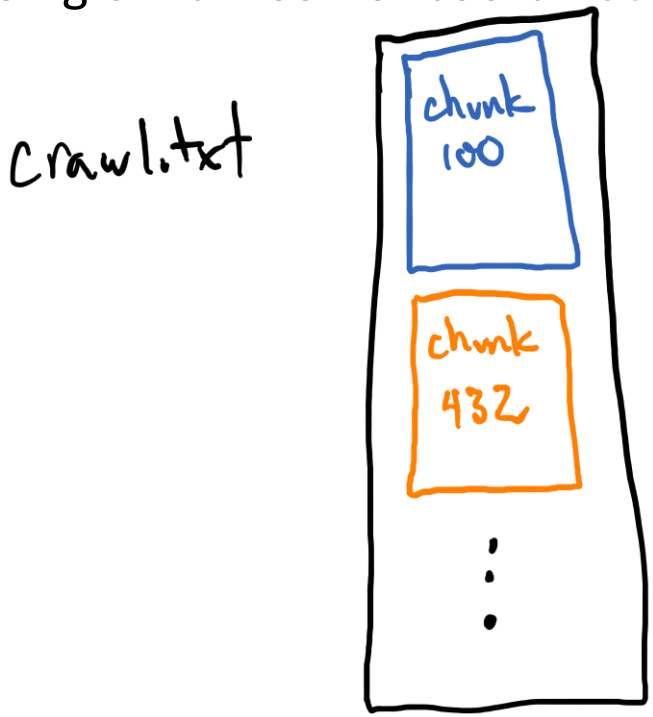
- Solution:
  - Store multiple copies of everything
  - Keep track of where those copies are

GFS design solution

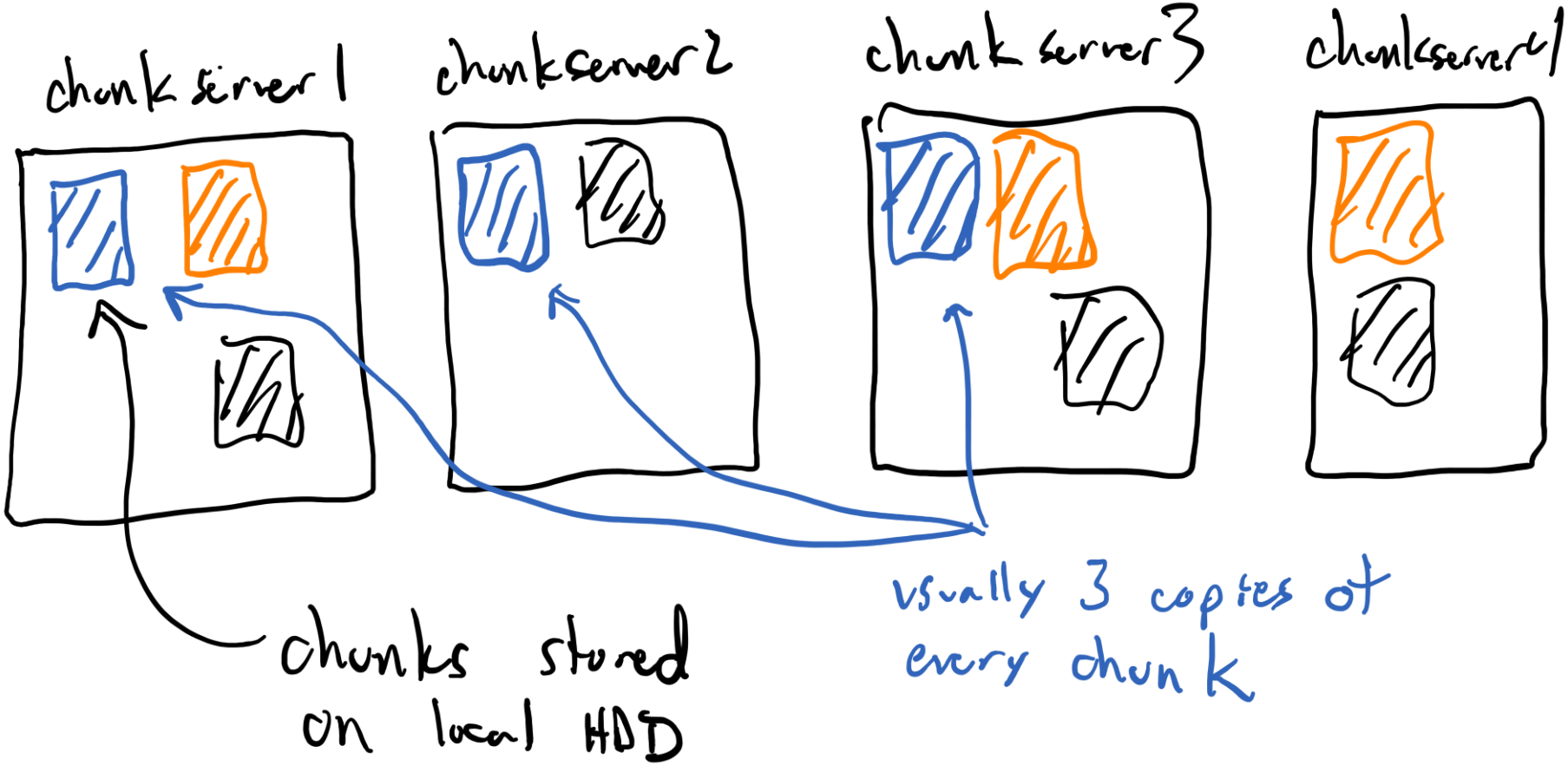
- Files stored as series of ***chunks***
- Reliability through replication
  - Each chunk replicated across 3+ ***servers***
- Single main server coordinates access, metadata

GFS design solution

- Files stored as series of ***chunks***
- Reliability through replication
  - Each chunk replicated across 3+ ***servers***
- Single main server coordinates access, metadata



Chunkservers



Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- GFS design
- **Read**
- Write
  - Consistency
- Append
- Fault tolerance
- Summary

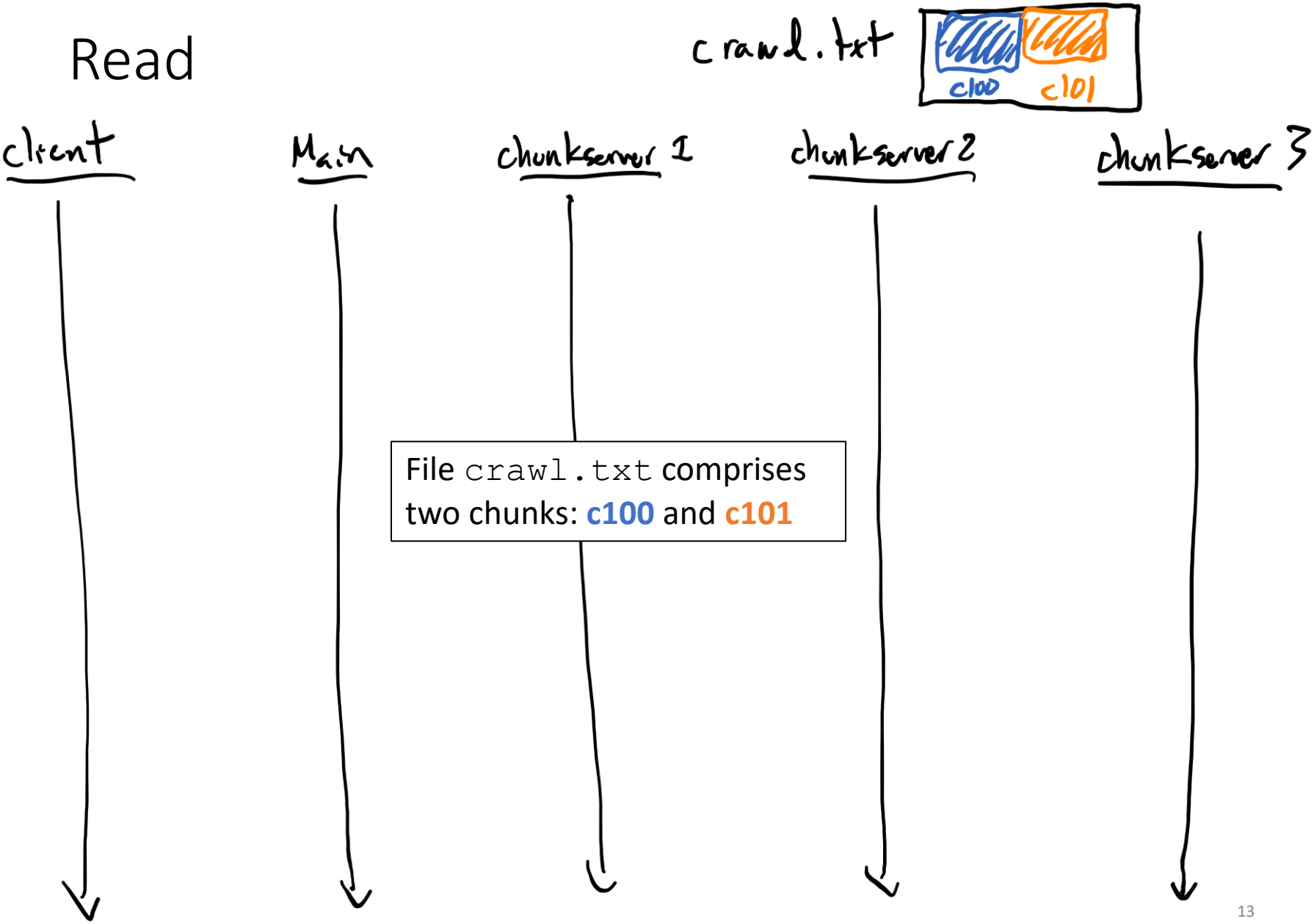
GFS file read

- Client asks Main for filename info
- Main responds with chunklist, and location(s) for each chunk
- Client fetches each chunk from a chunkserver

GFS file read example

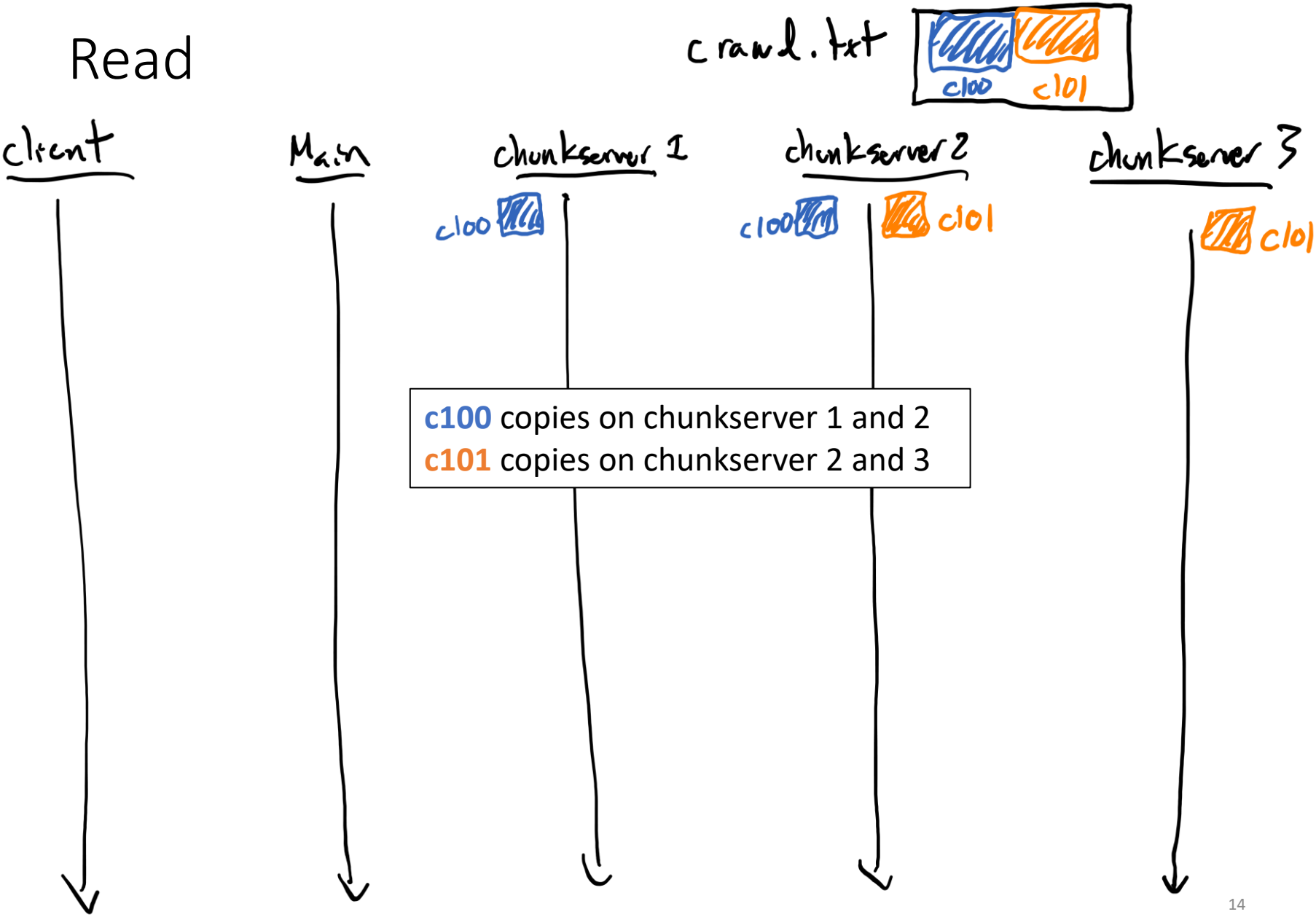
- One read
- One file crawl.txt
- 2 chunks
- 3 chunk servers
- 2 copies of each chunk (replicas)

Read



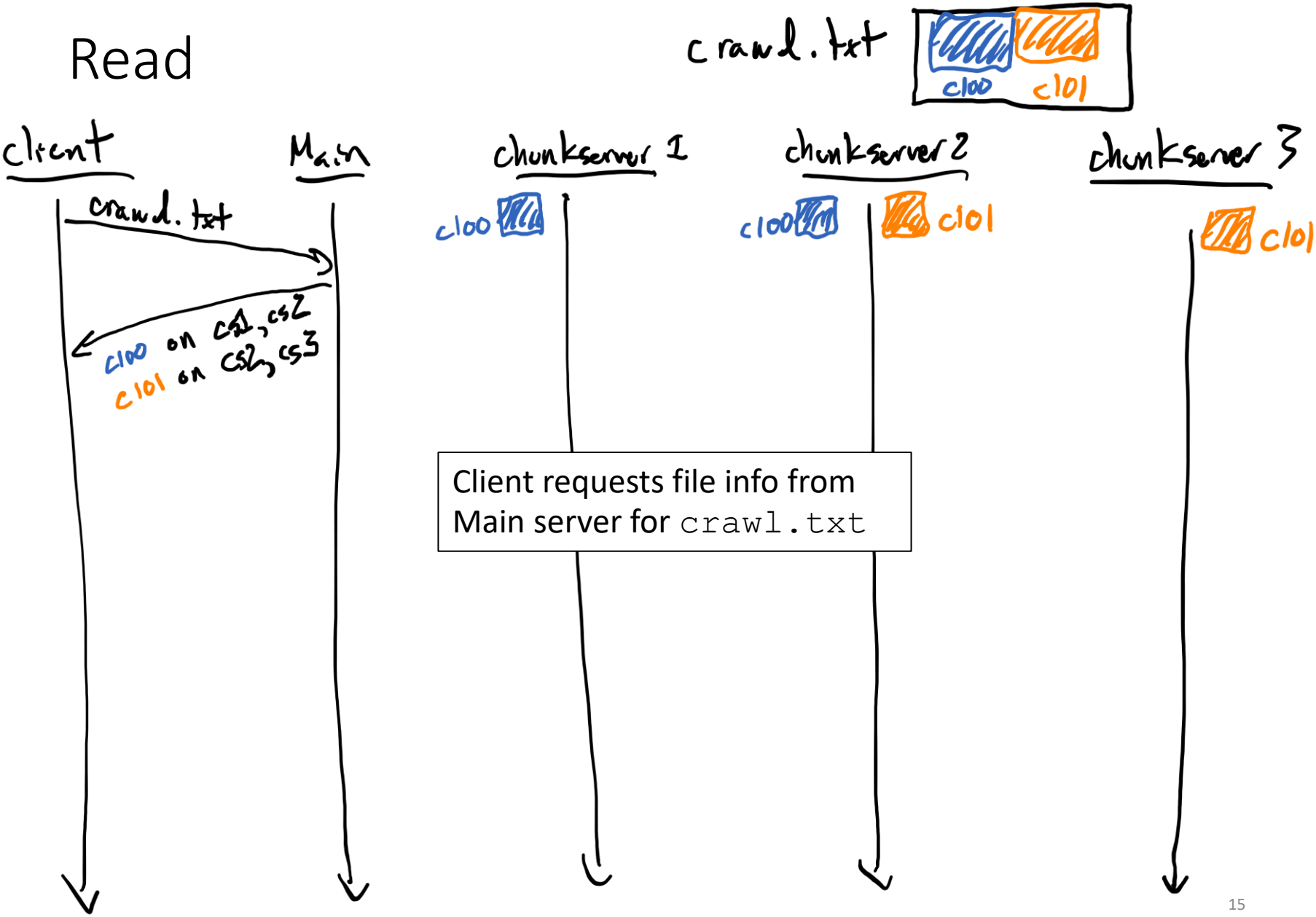
File `crawl.txt` comprises two chunks: **c100** and **c101**

Read



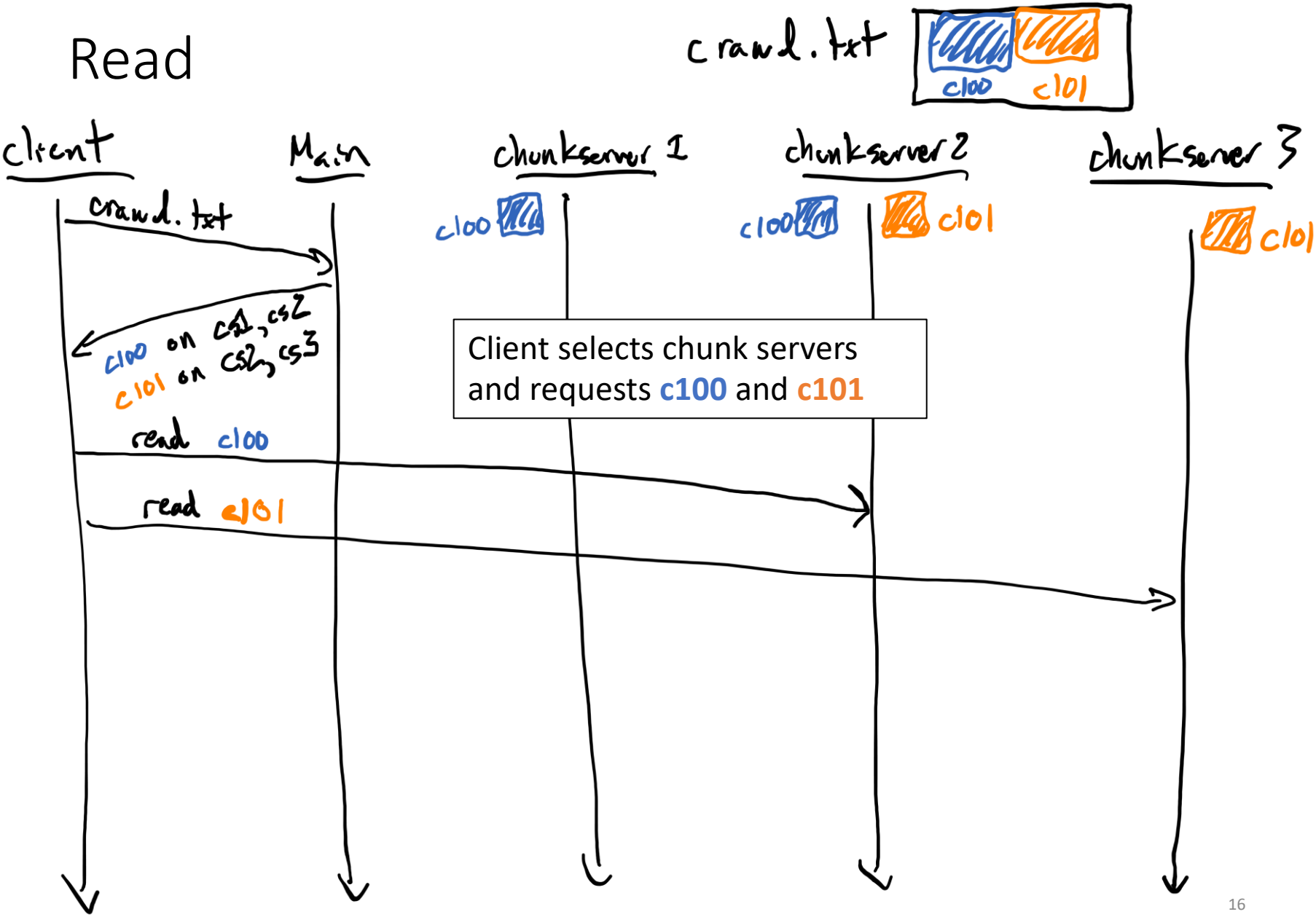
c100 copies on chunkserver 1 and 2
c101 copies on chunkserver 2 and 3

Read



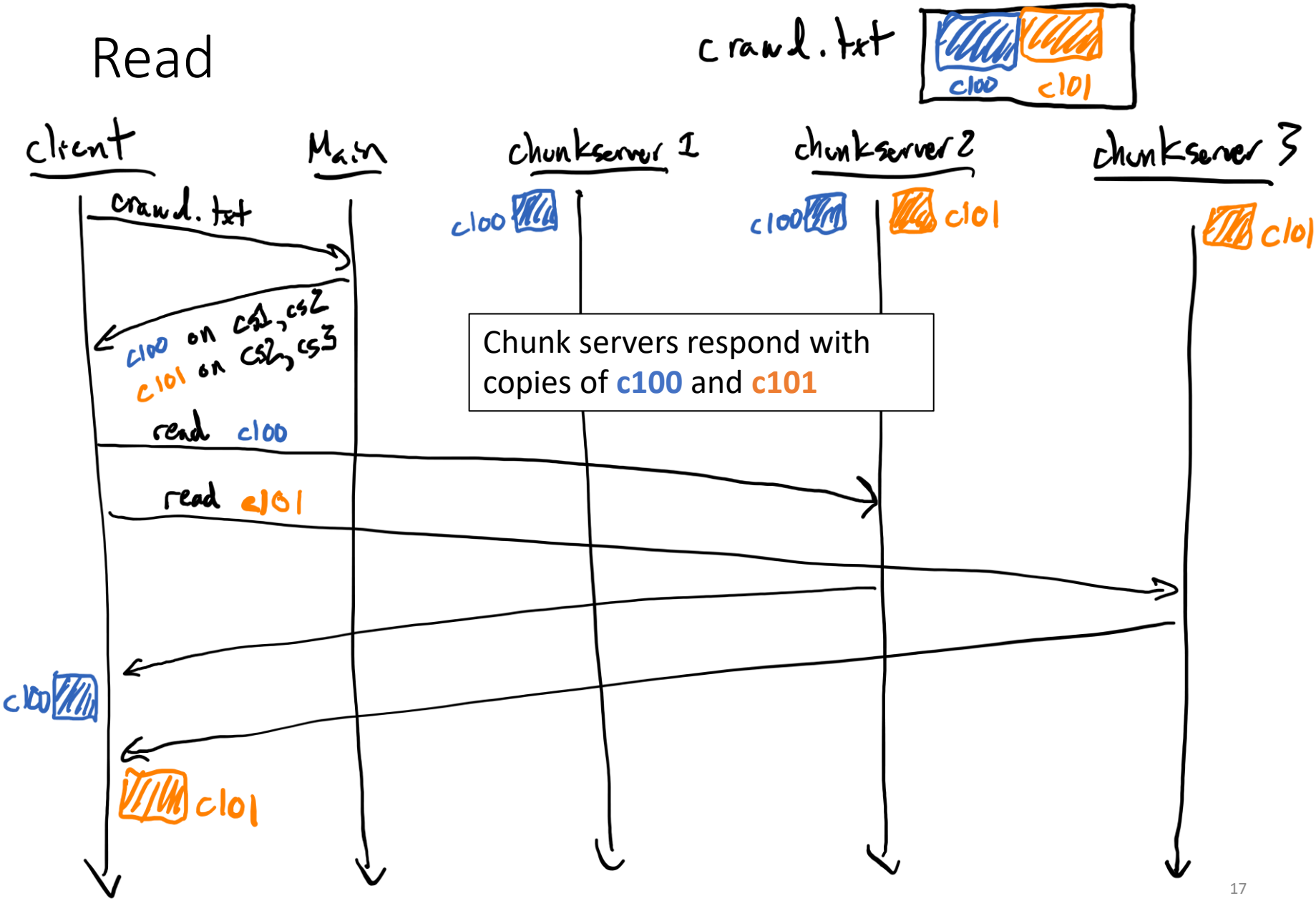
Client requests file info from Main server for `crawl.txt`

Read



Client selects chunk servers and requests **c100** and **c101**

Read



Chunk servers respond with copies of **c100** and **c101**

Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- GFS design
- Read
- **Write**
  - **GFS Consistency**
  - **GFS Defined**
- Append
- Fault tolerance
- Summary

GFS file write

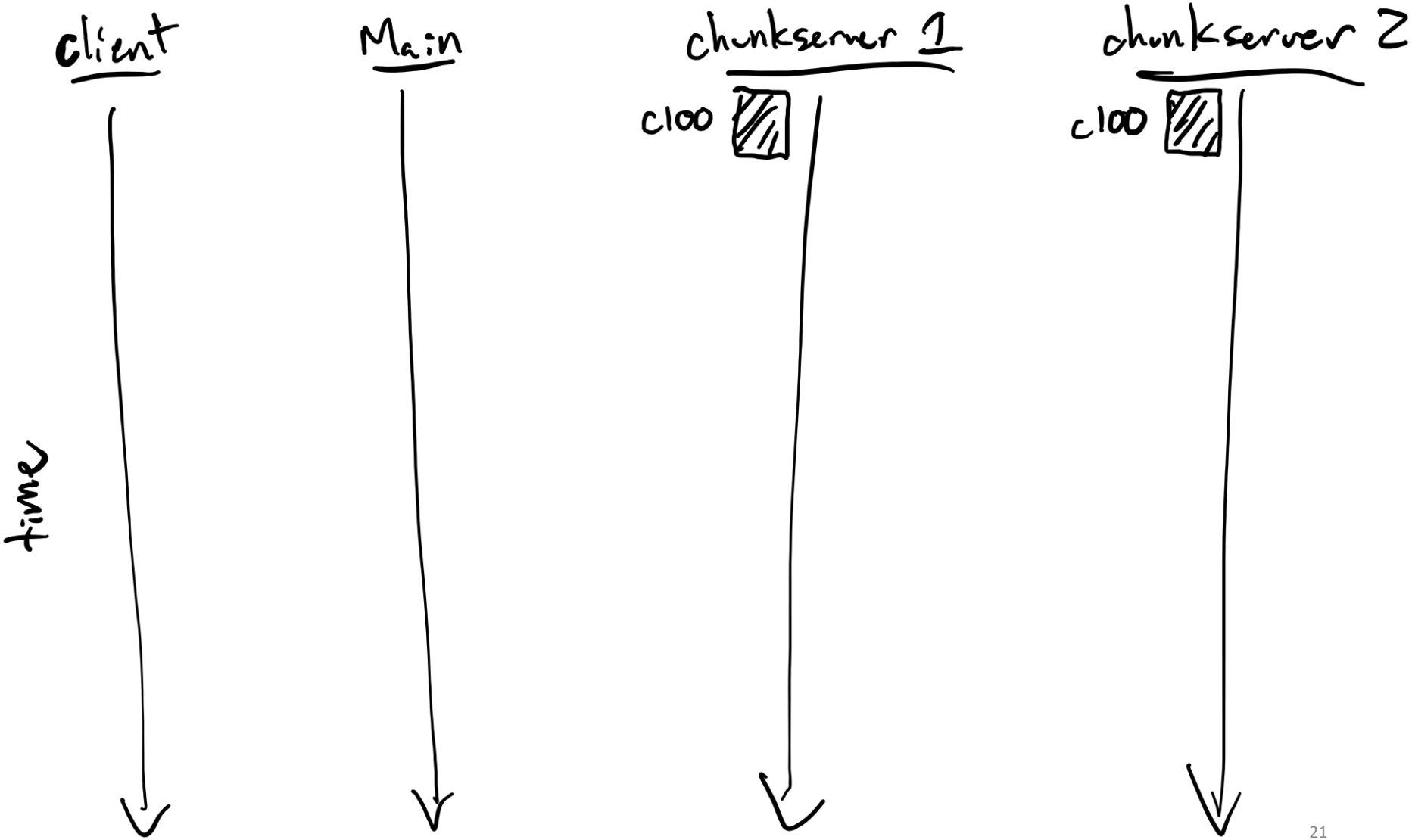
- A write would work just like a read if
  - Clients take turns; and
  - There are no errors
- Serial, successful writes
- Spoiler alert: these assumptions break down

One write

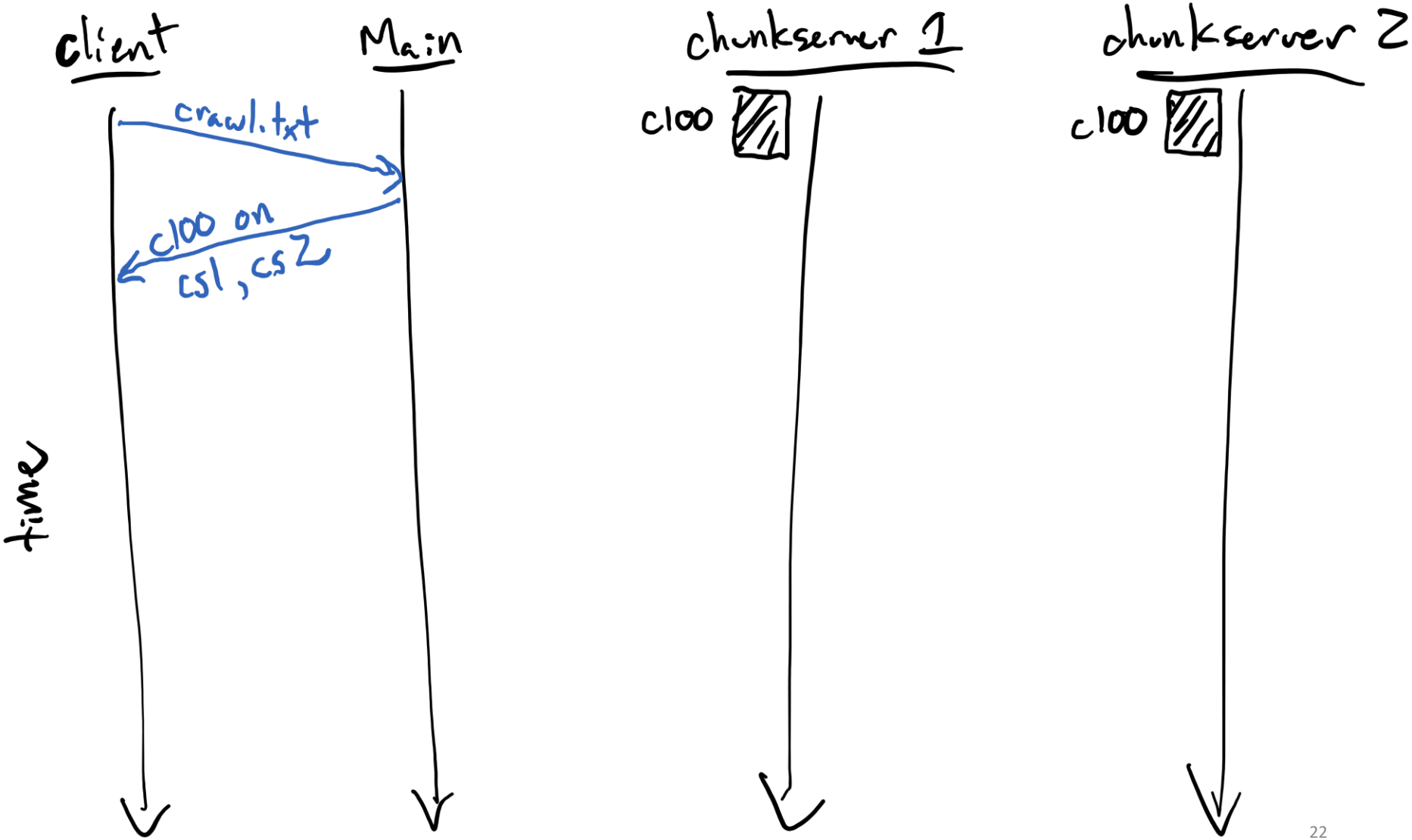
- One write
- All chunkservers get the same value for the same chunk

- We'll start with a very small example
  - One file crawl.txt
  - One chunk
  - Two chunk servers

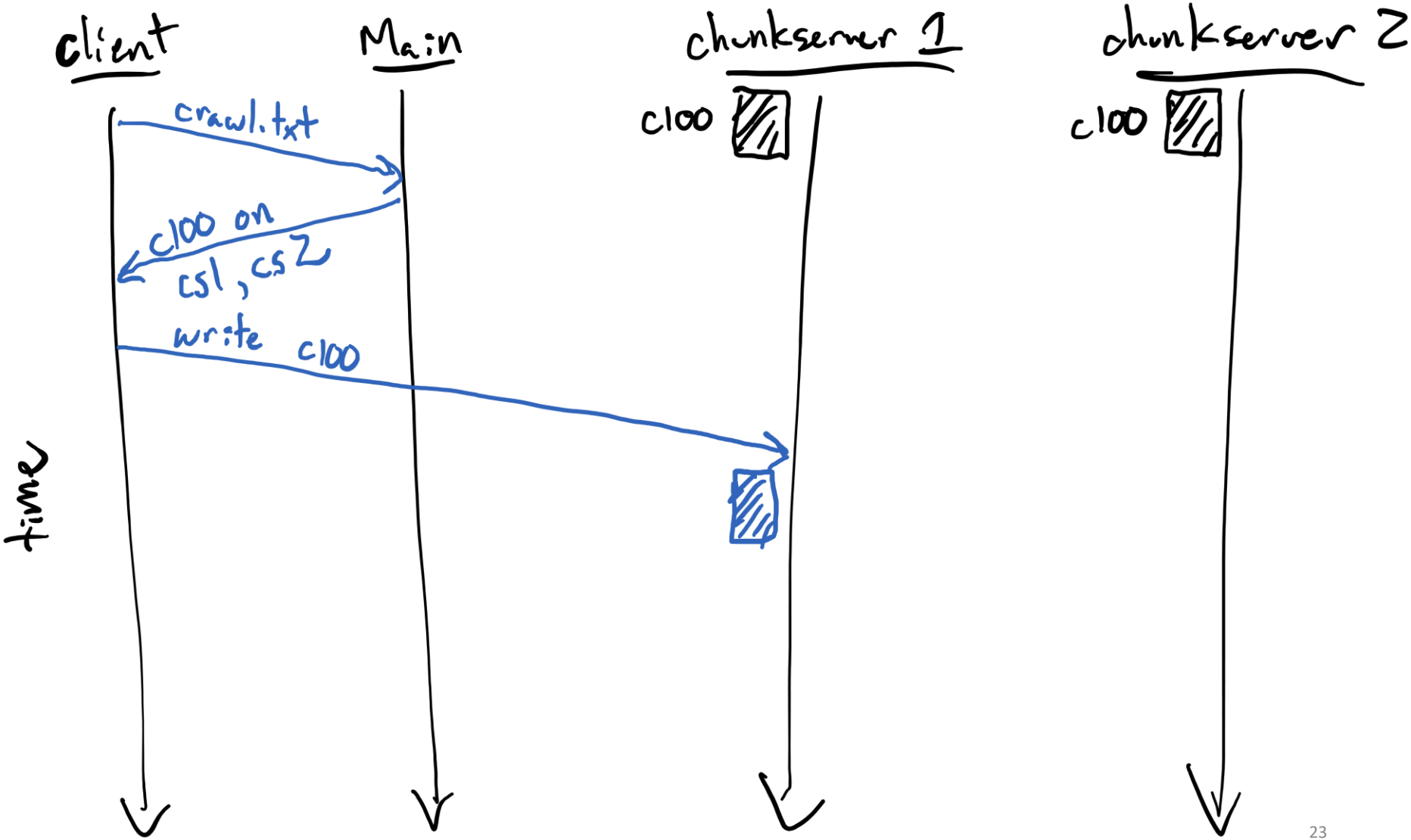
One write



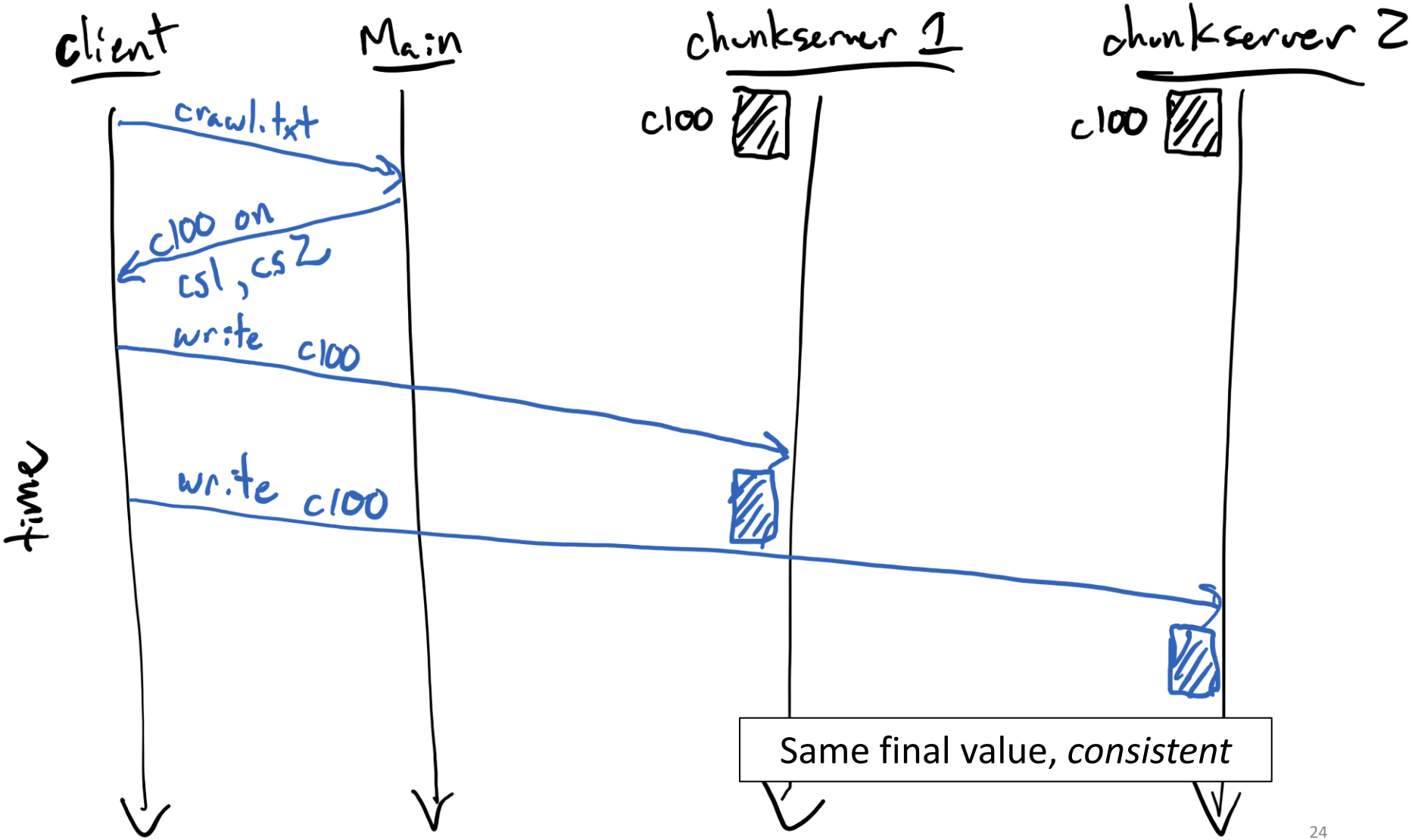
One write


client
Main
chunkserver 1
chunkserver 2
crawl.txt
c100 on
cs1, cs2
c100
c100
time

One write


client
Main
chunkserver 1
chunkserver 2
c100
c100
crawl.txt
c100 on
cs1, cs2
write c100
time

One write

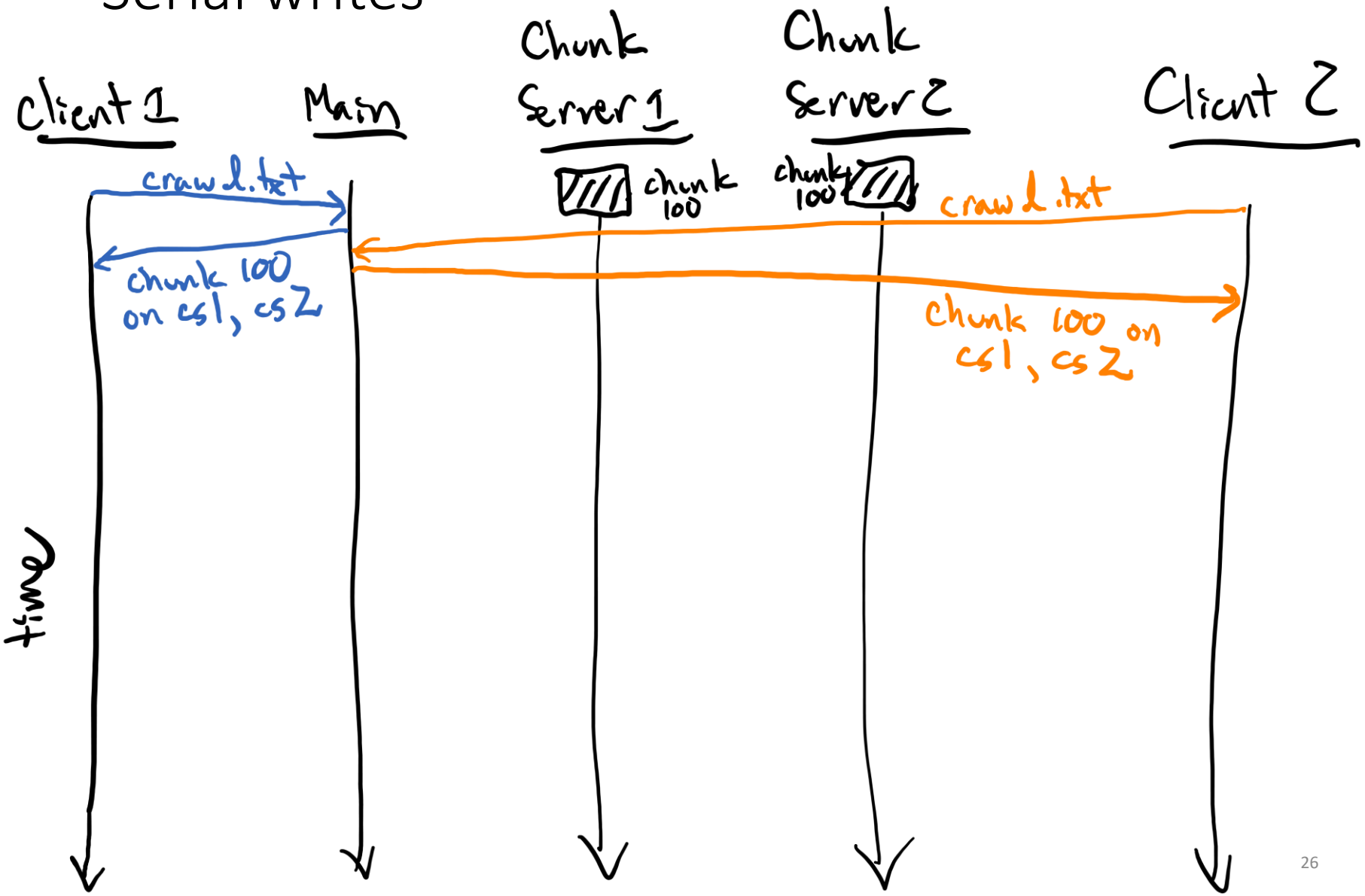


Same final value, *consistent*

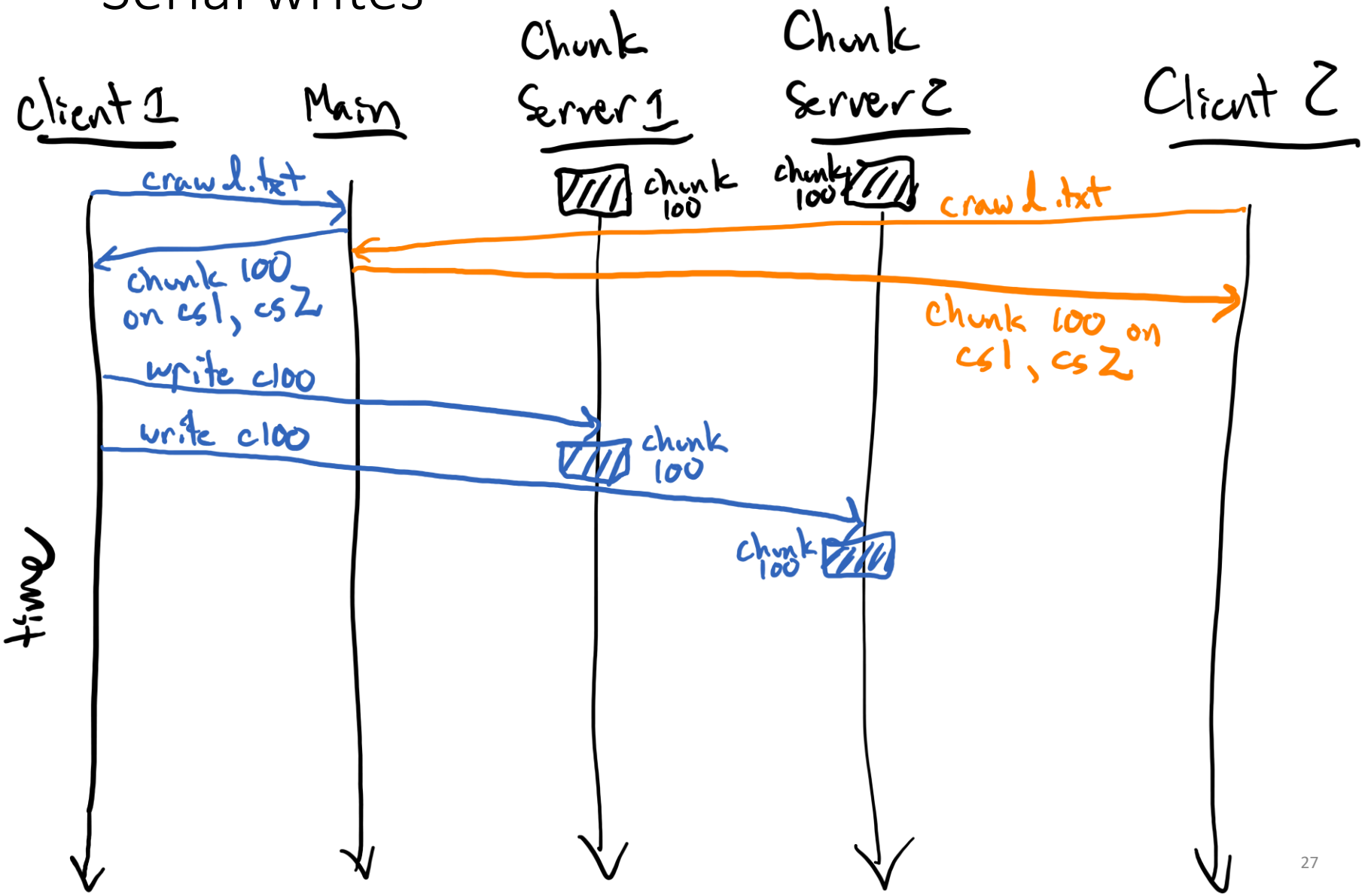
Serial writes

- Two serial writes
- First write
  - All chunkservers get the same value for the same chunk
- Second write
  - All chunkservers get the same new value for the same chunk

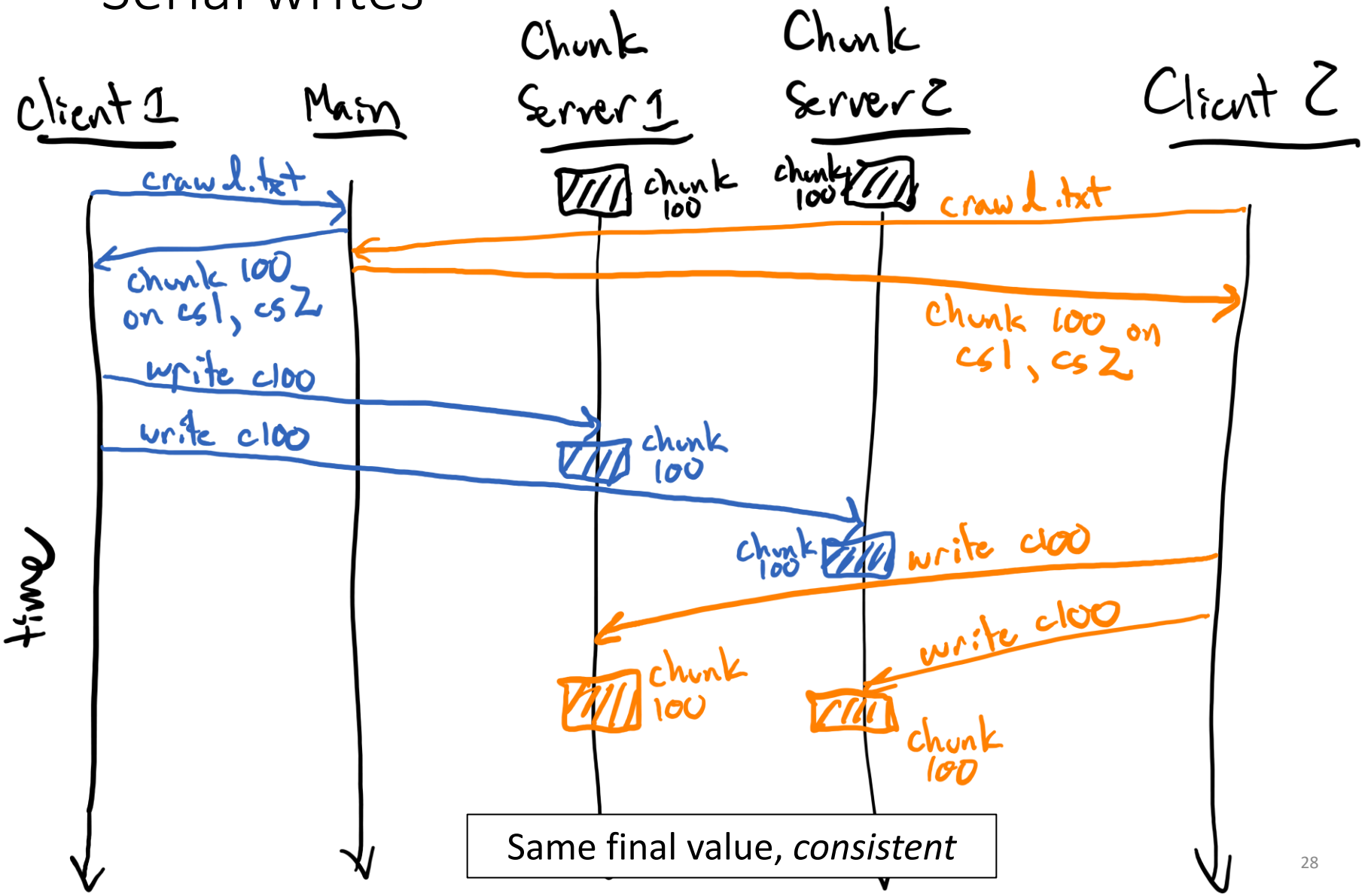
Serial writes



Serial writes



Serial writes



Same final value, *consistent*

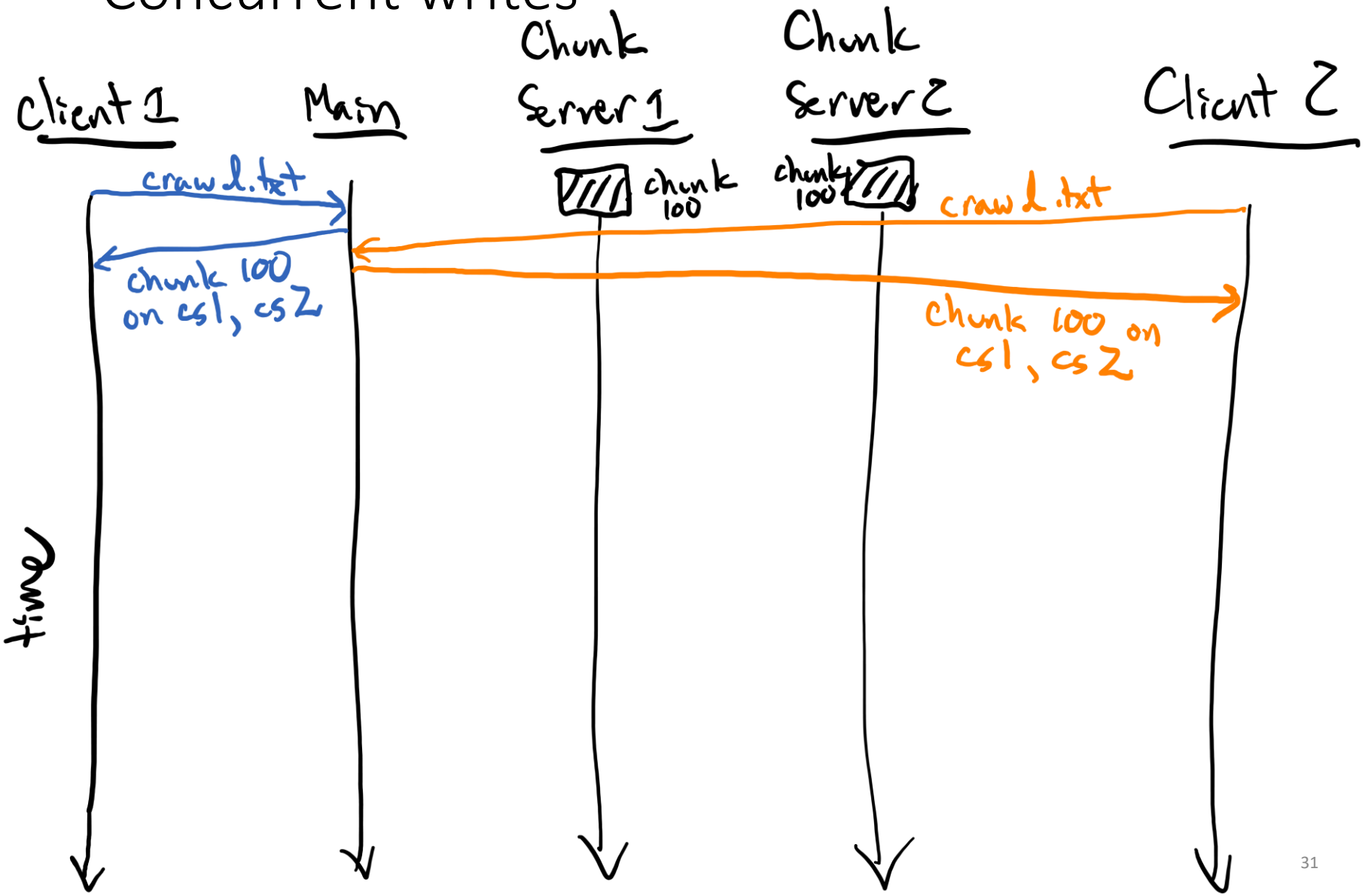
Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- GFS design
- Read
- Write
  - **GFS Consistent**
  - GFS Defined
- Append
- Fault tolerance
- Summary

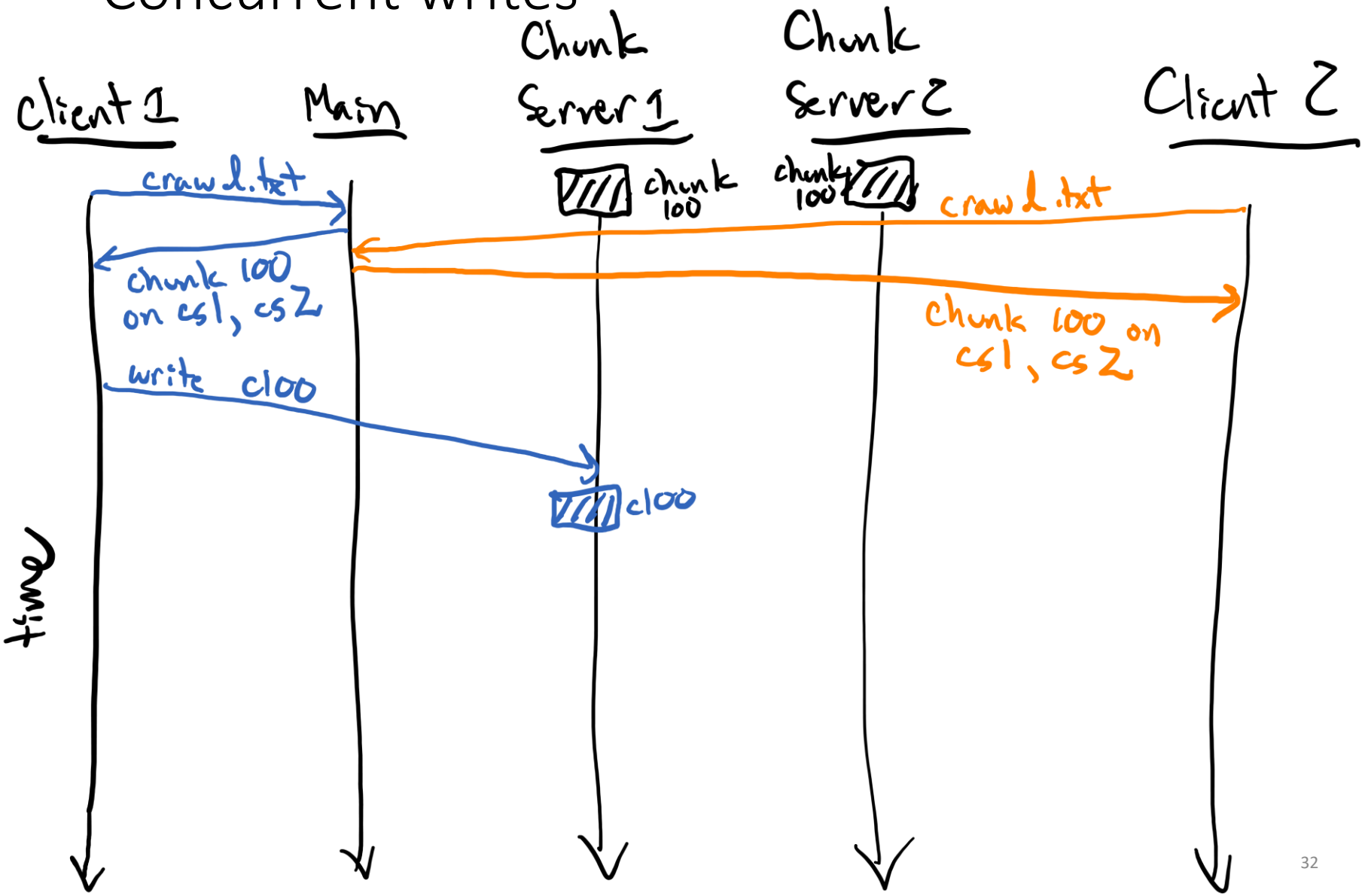
Concurrent writes

- Two writes from two clients happen at (almost) the same time
- Problem: two different values on two different chunkservers for the same chunk
  - *Inconsistent*

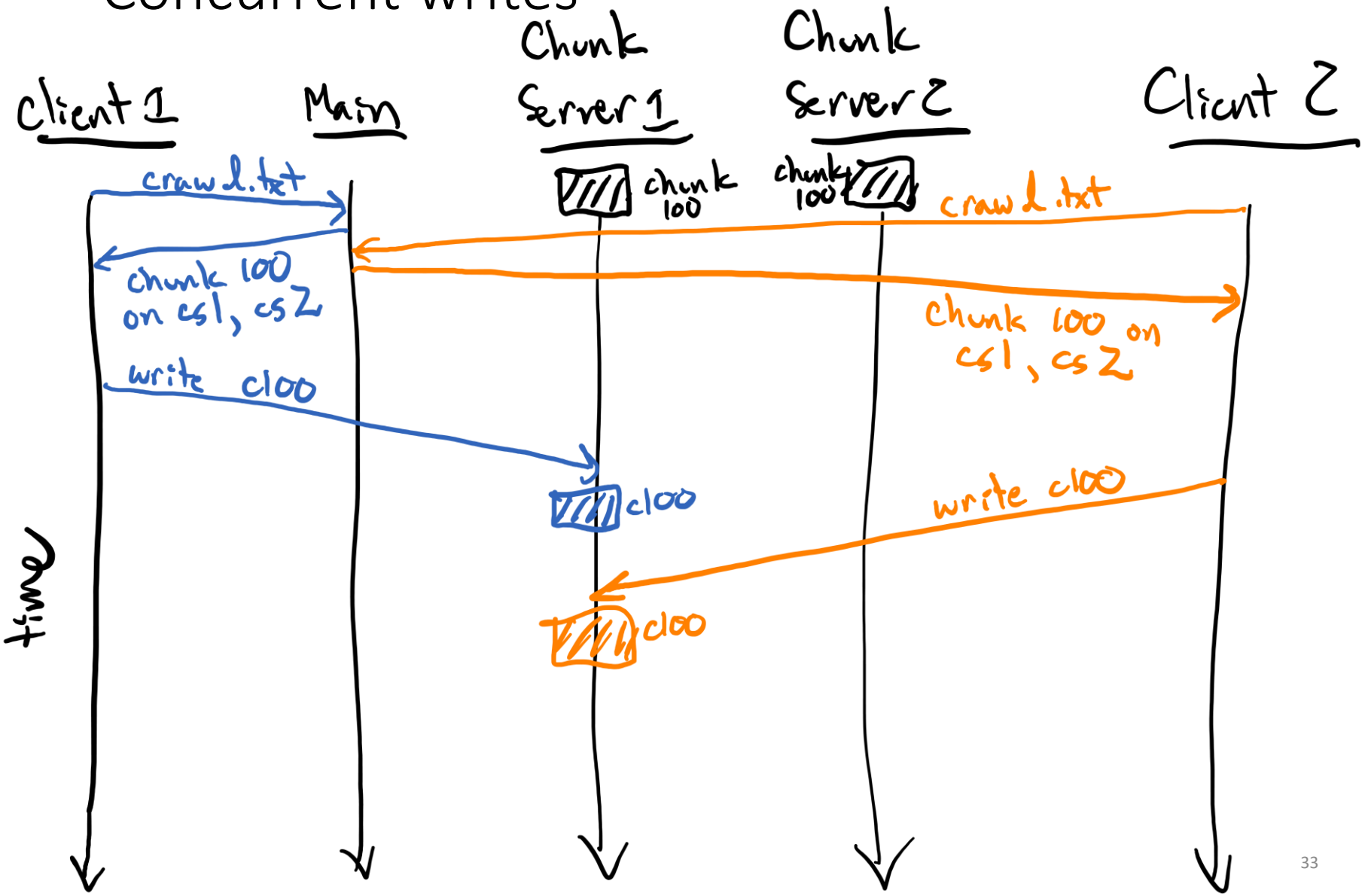
Concurrent writes



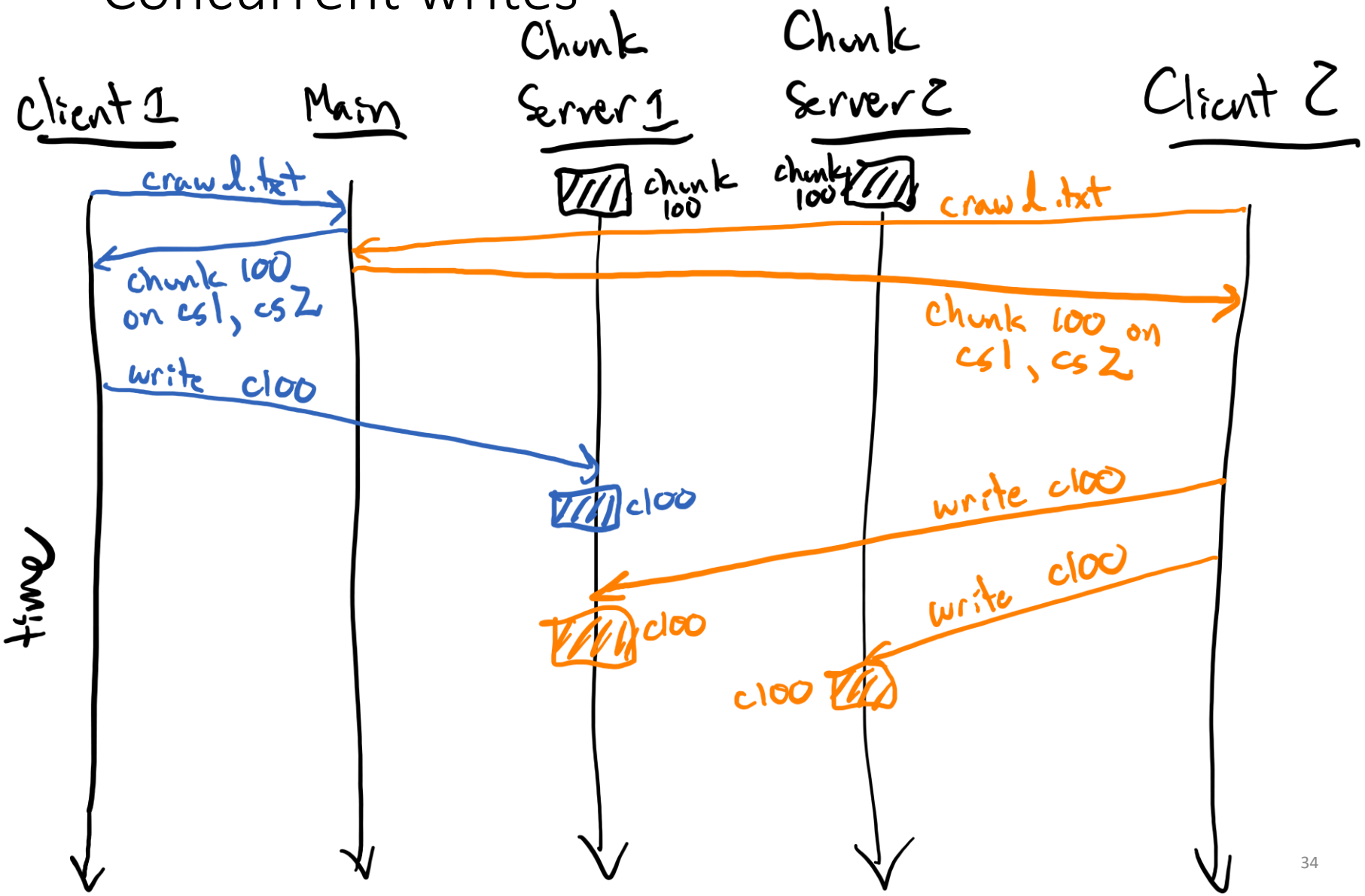
Concurrent writes



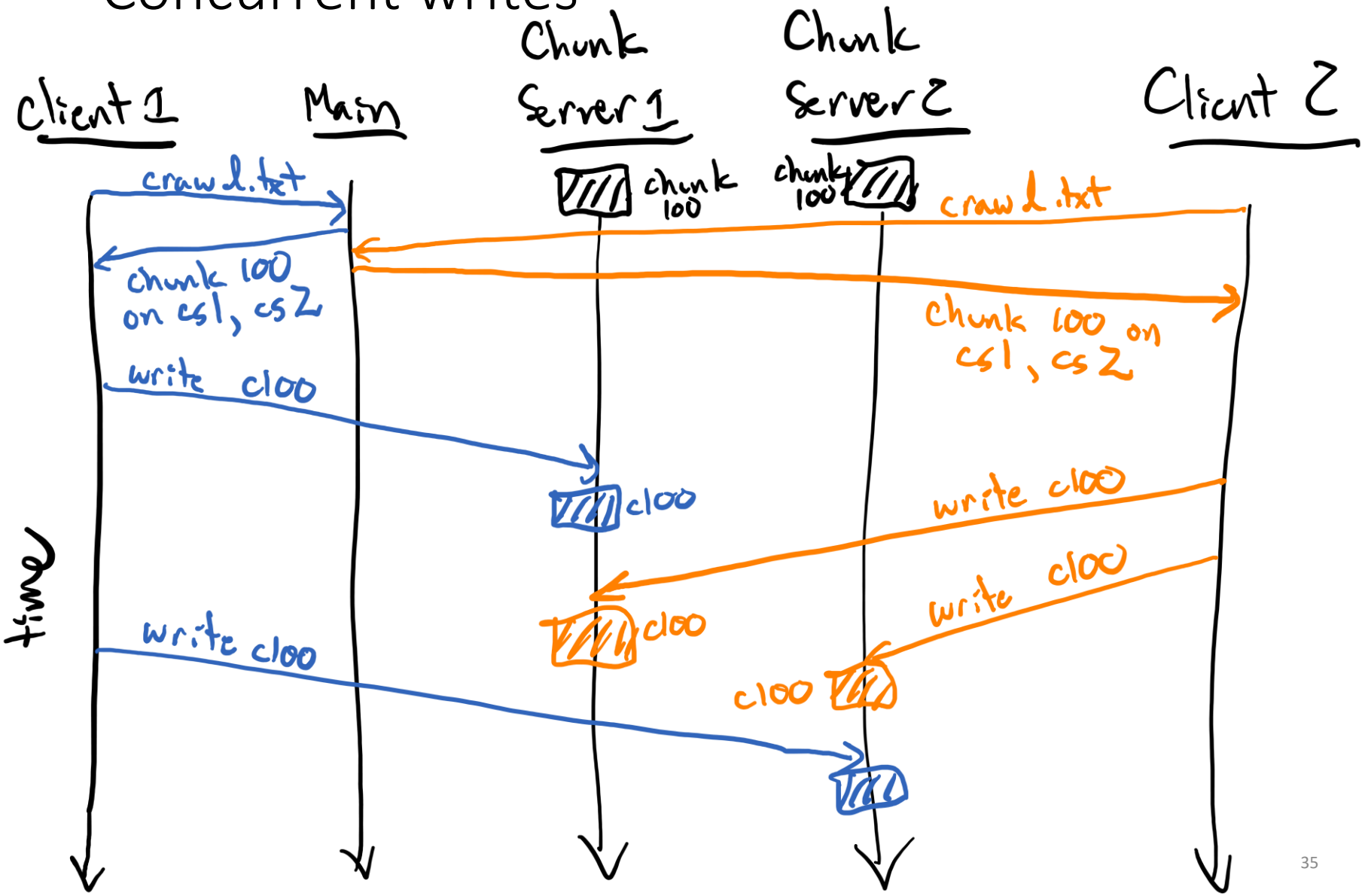
Concurrent writes



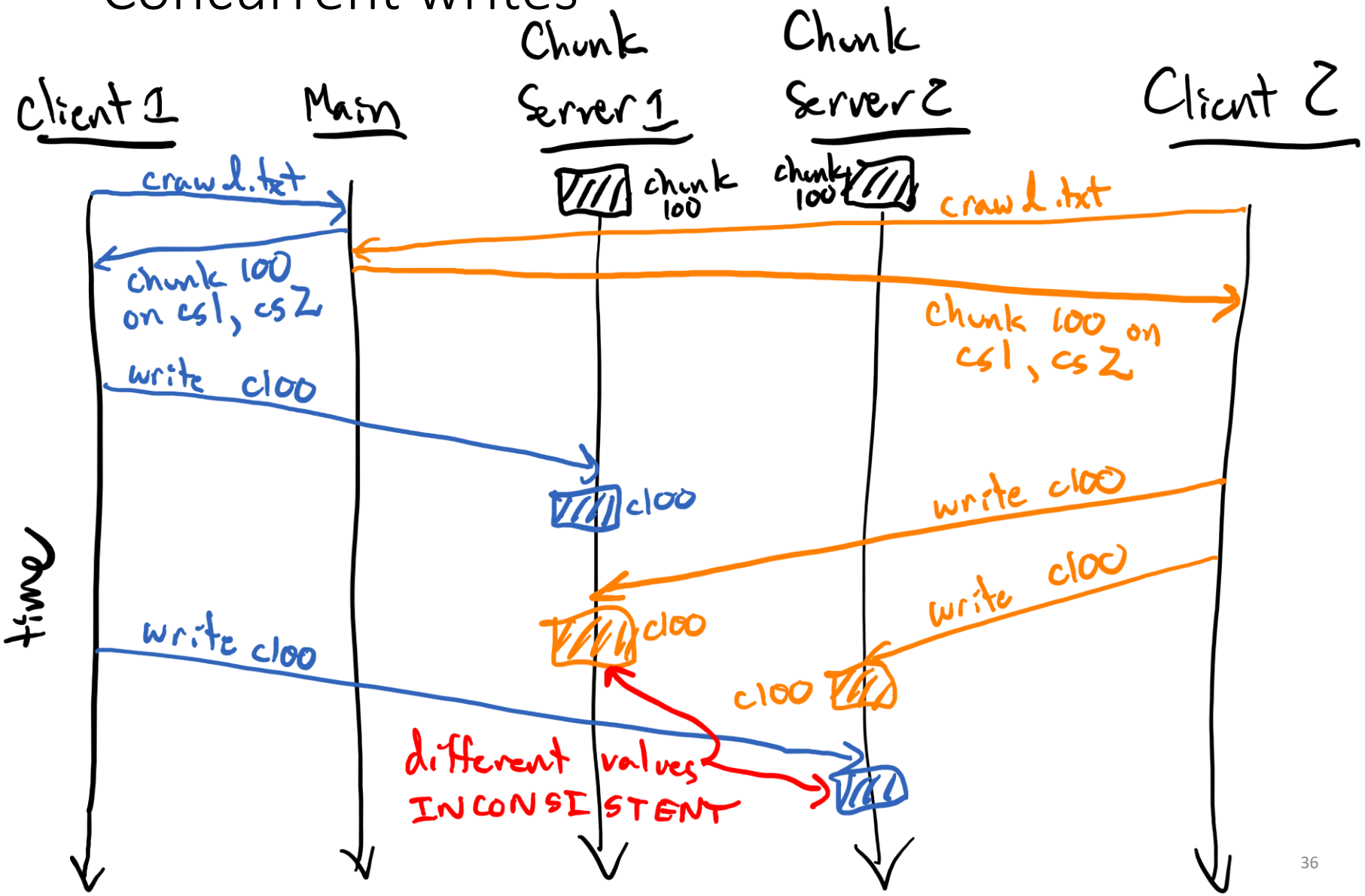
Concurrent writes



Concurrent writes



Concurrent writes



GFS write forwarding

- *Consistent* in GFS: Every chunkserver has the same value for the same chunk

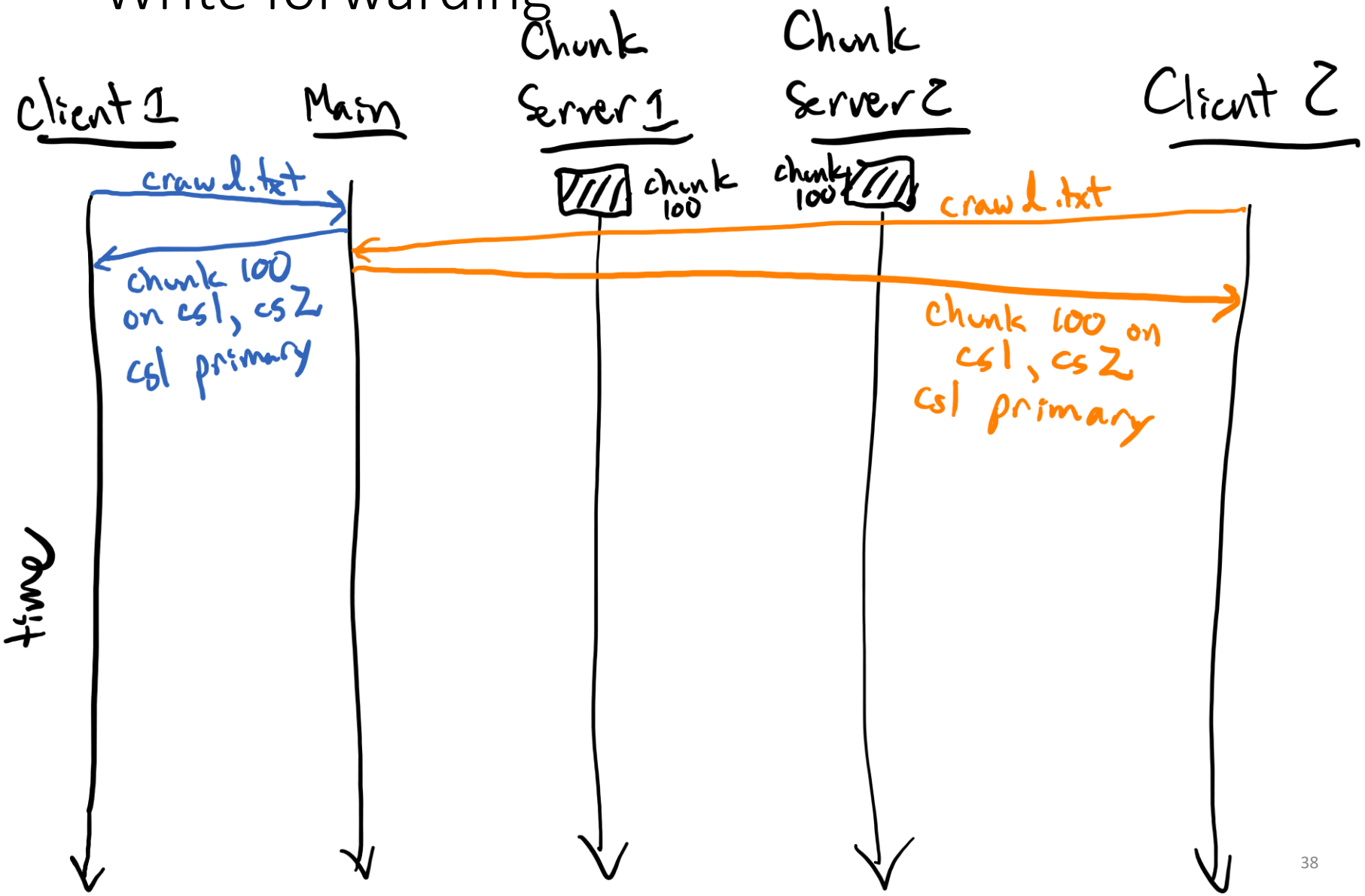
- To guarantee consistency, writes need to be coordinated: replicas receive updates in the same order

- GFS calls this *write forwarding*

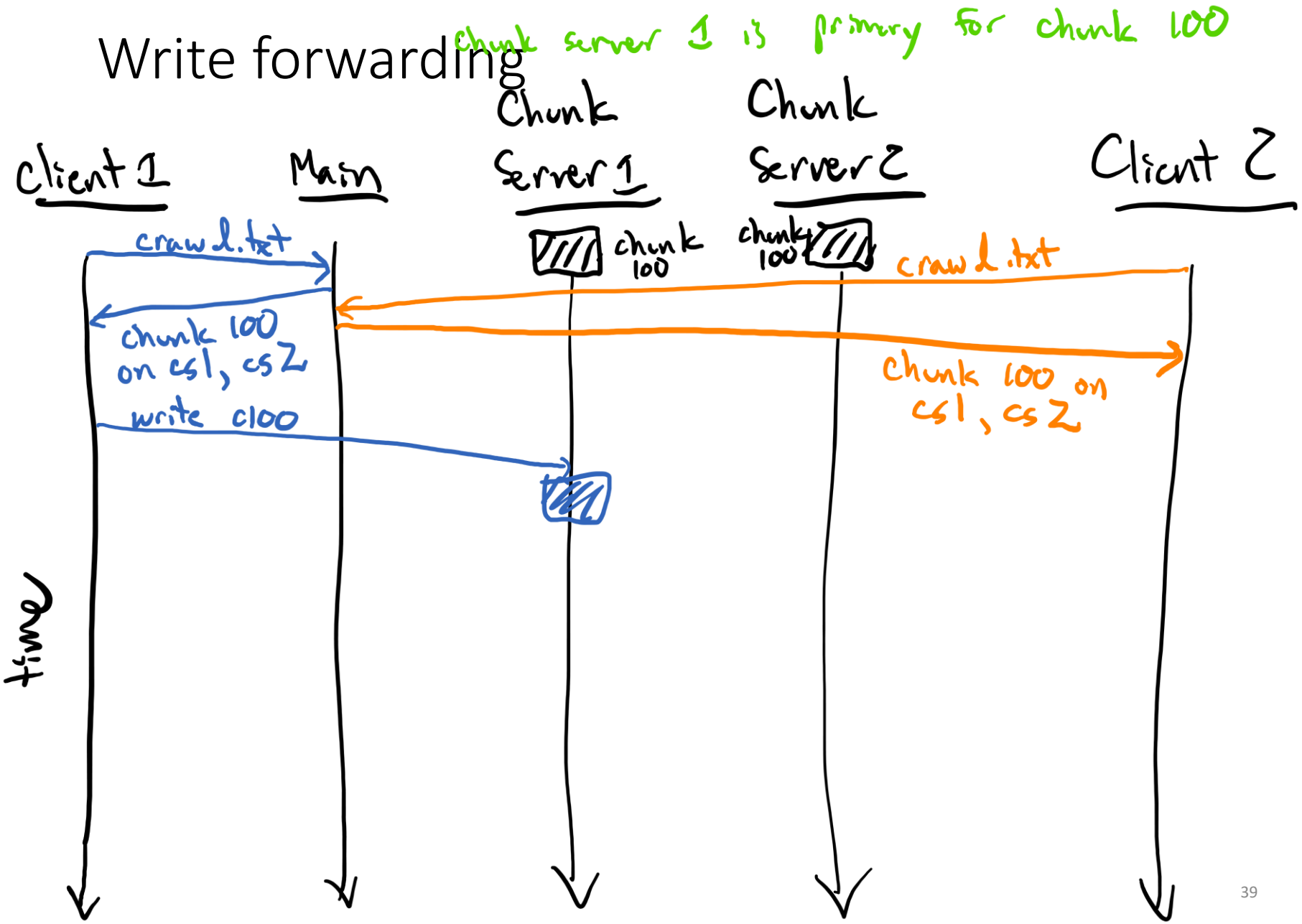
- One chunkserver is designated as *primary* for each chunk id
- Primary forwards writes to replica chunkservers*

*This is a simplification, see the paper for full details.
http://static.googleusercontent.com/media/research.google.com/en//archive/gfs-sosp2003.pdf

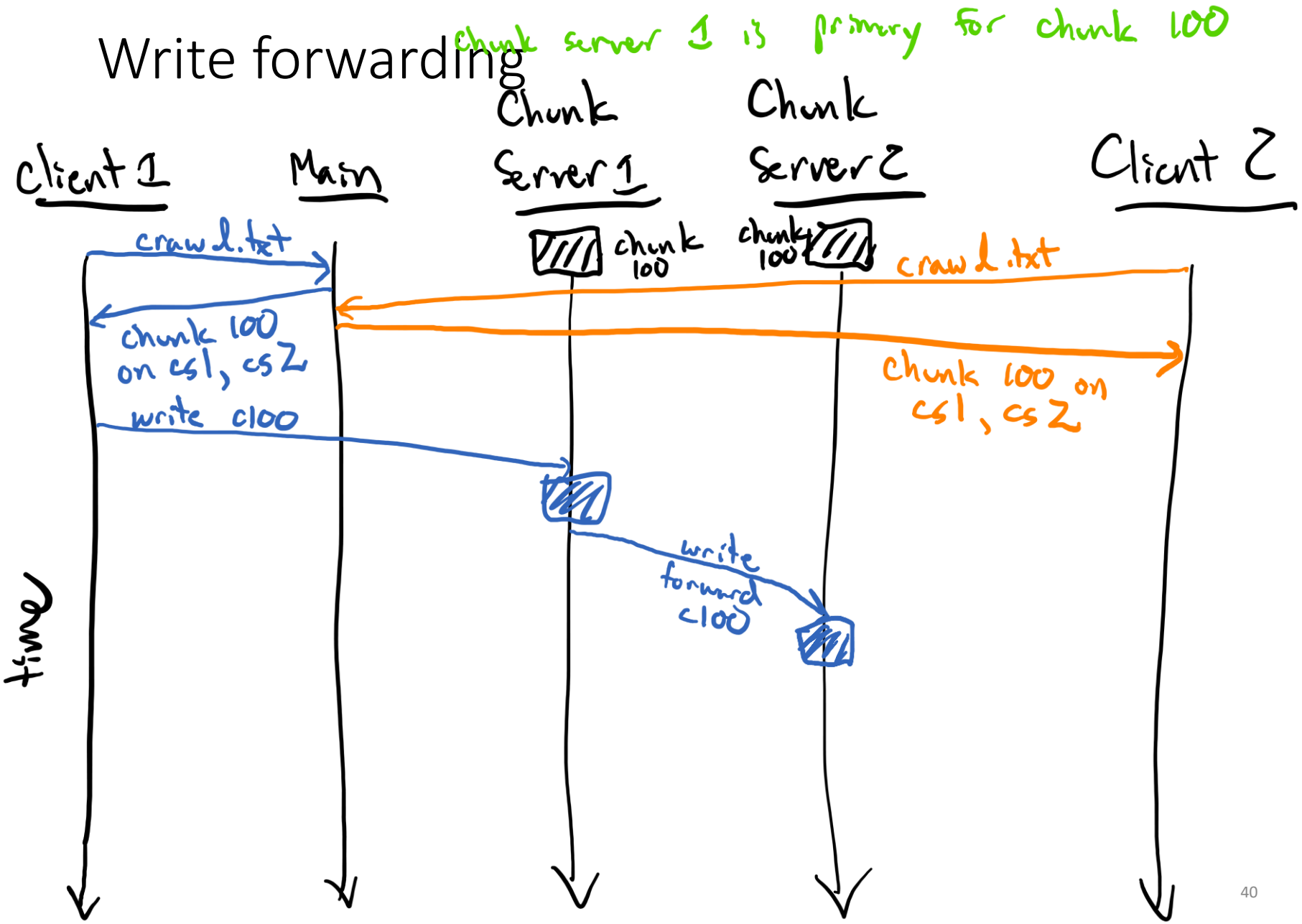
Write forwarding



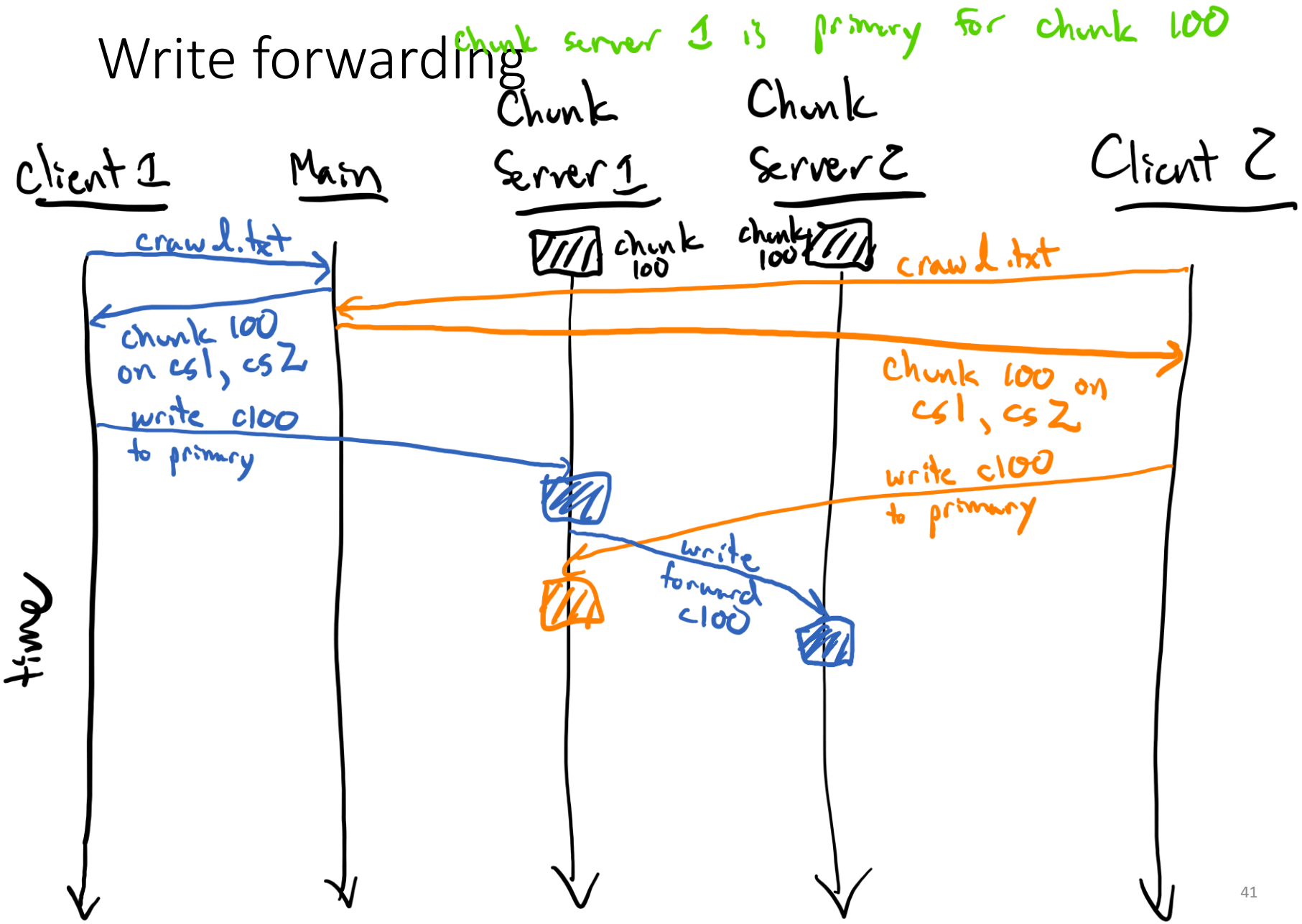
Write forwarding



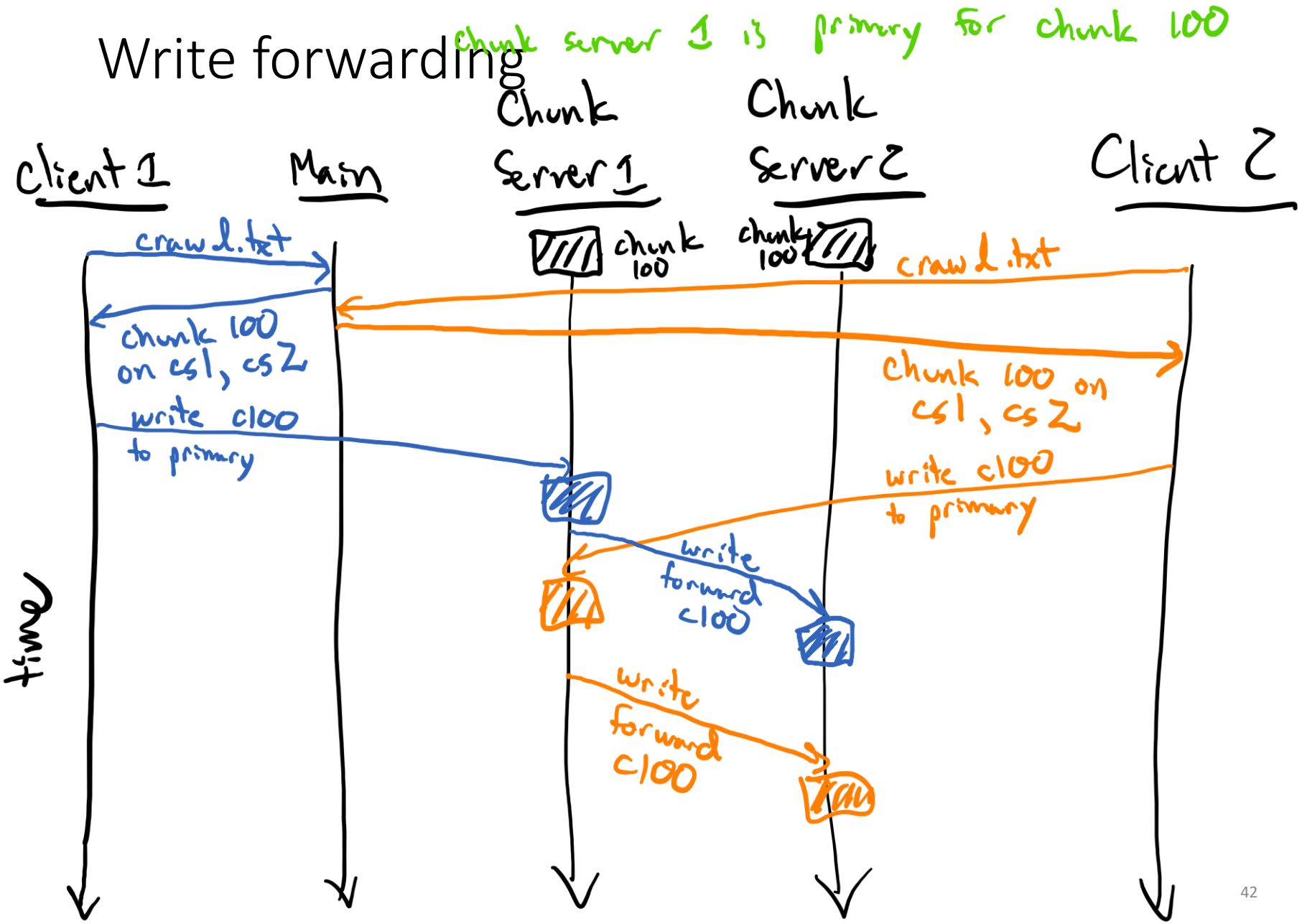
Write forwarding



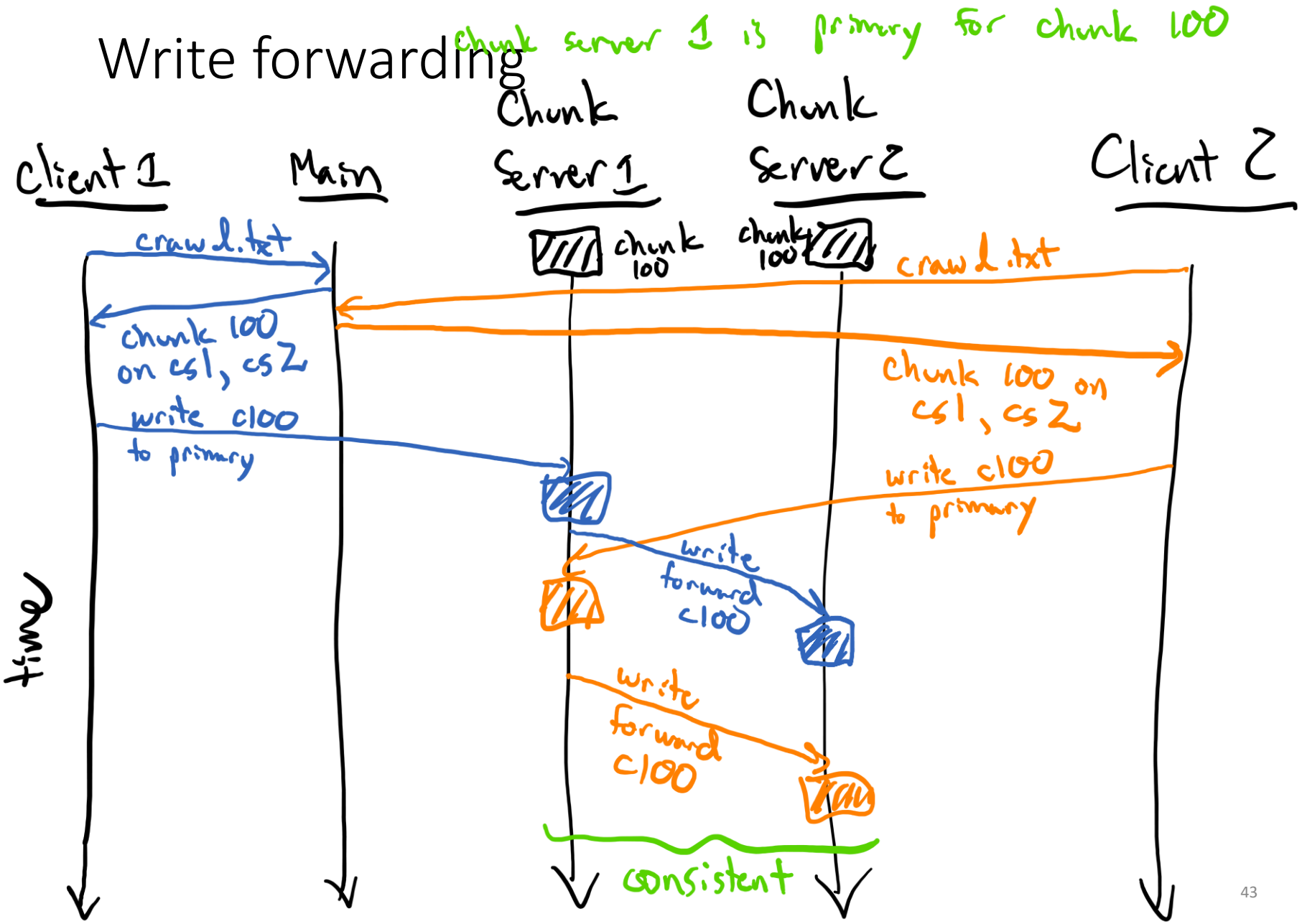
Write forwarding



Write forwarding



Write forwarding



Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- GFS design
- Read
- Write
  - GFS Consistent
  - **GFS Defined**
- Append
- Fault tolerance
- Summary

Properties: consistent, defined

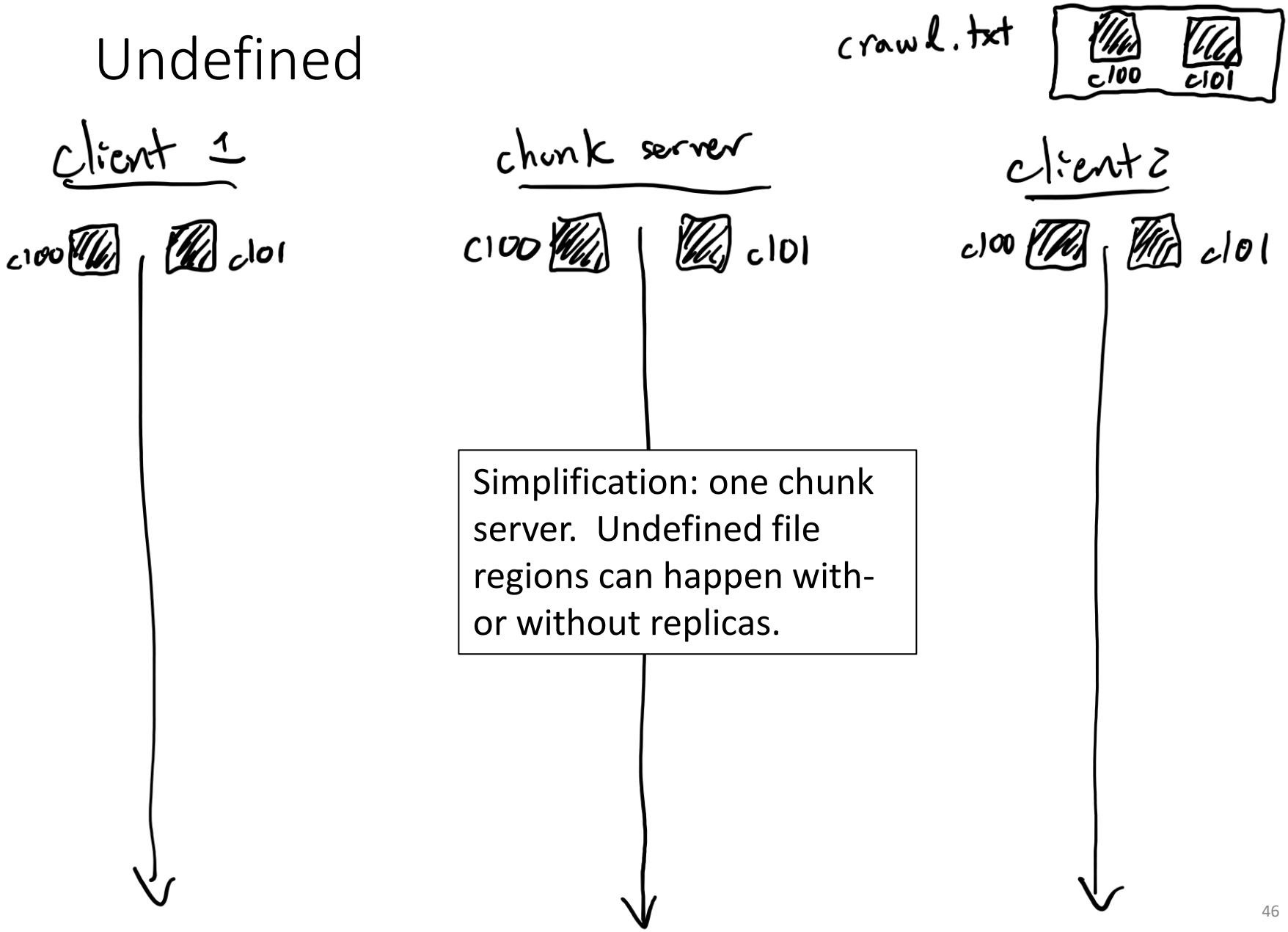
- *GFS Consistent*: Every chunk server eventually has the same value for the same chunk (updates in same order)

- *GFS Defined*: changes to different chunks of the same file from different writers did not get mixed together

- How do undefined values happen?

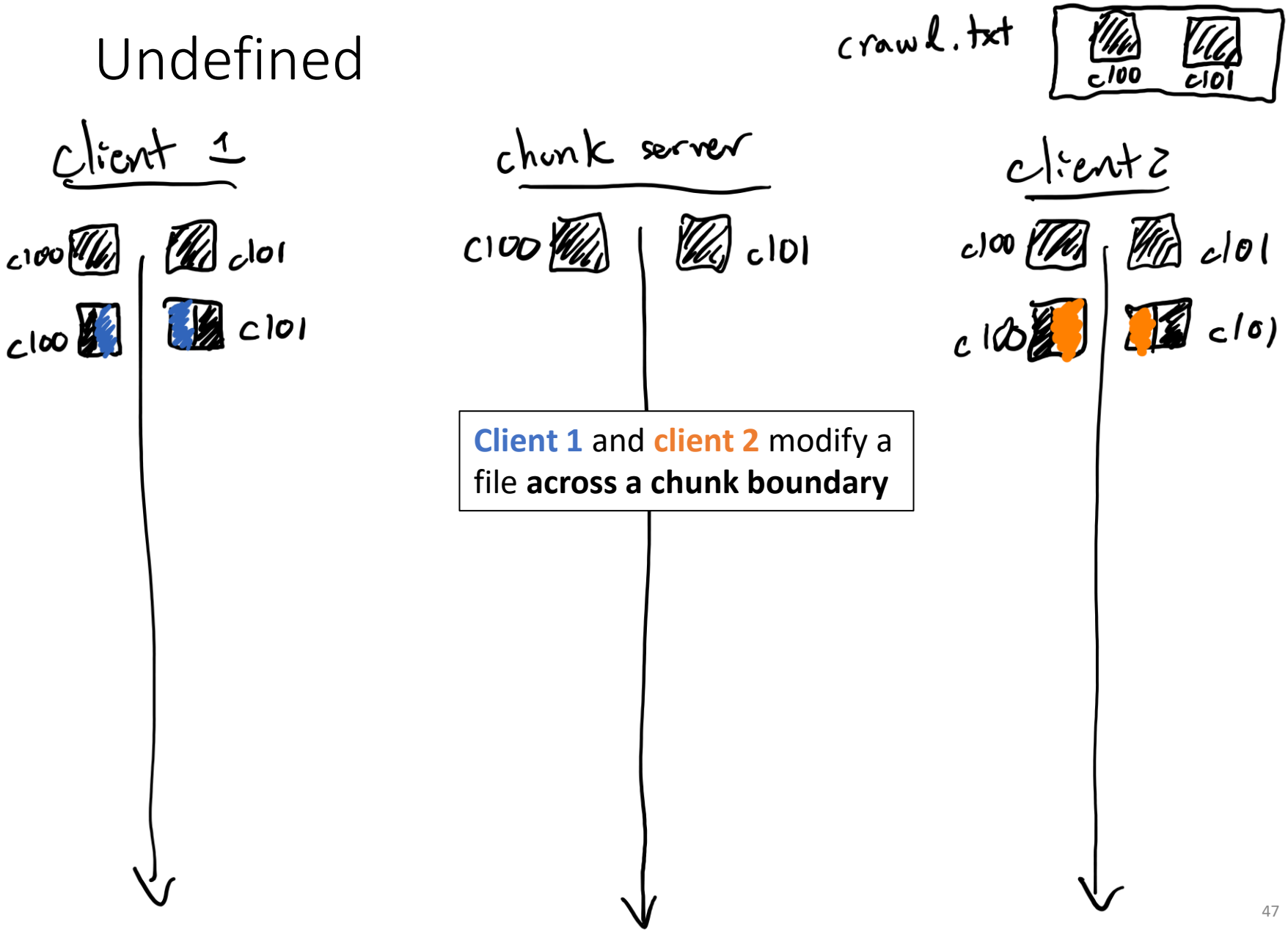
Simultaneous writes to different chunks in the same region of one file

Undefined

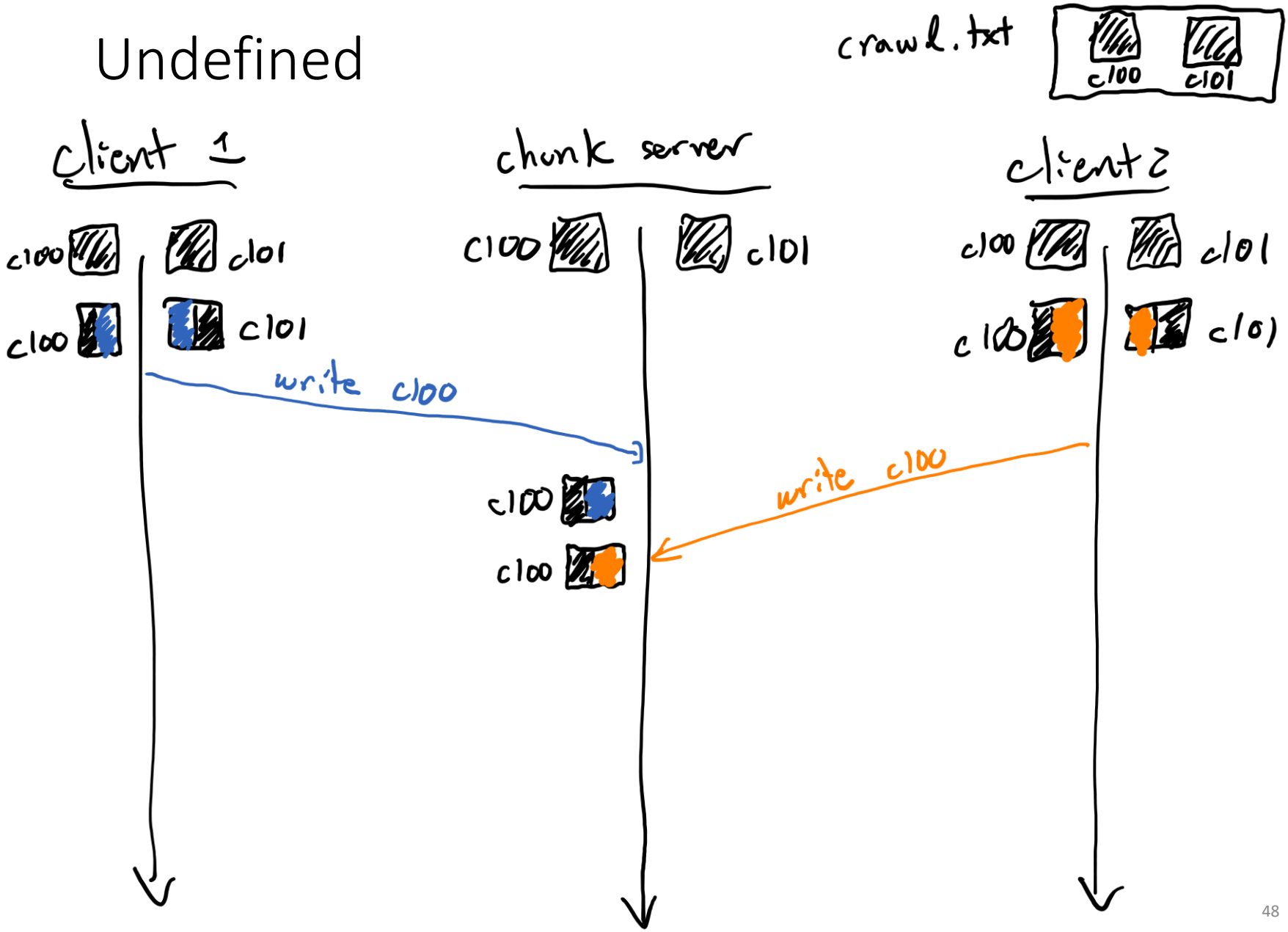


Simplification: one chunk server. Undefined file regions can happen with- or without replicas.

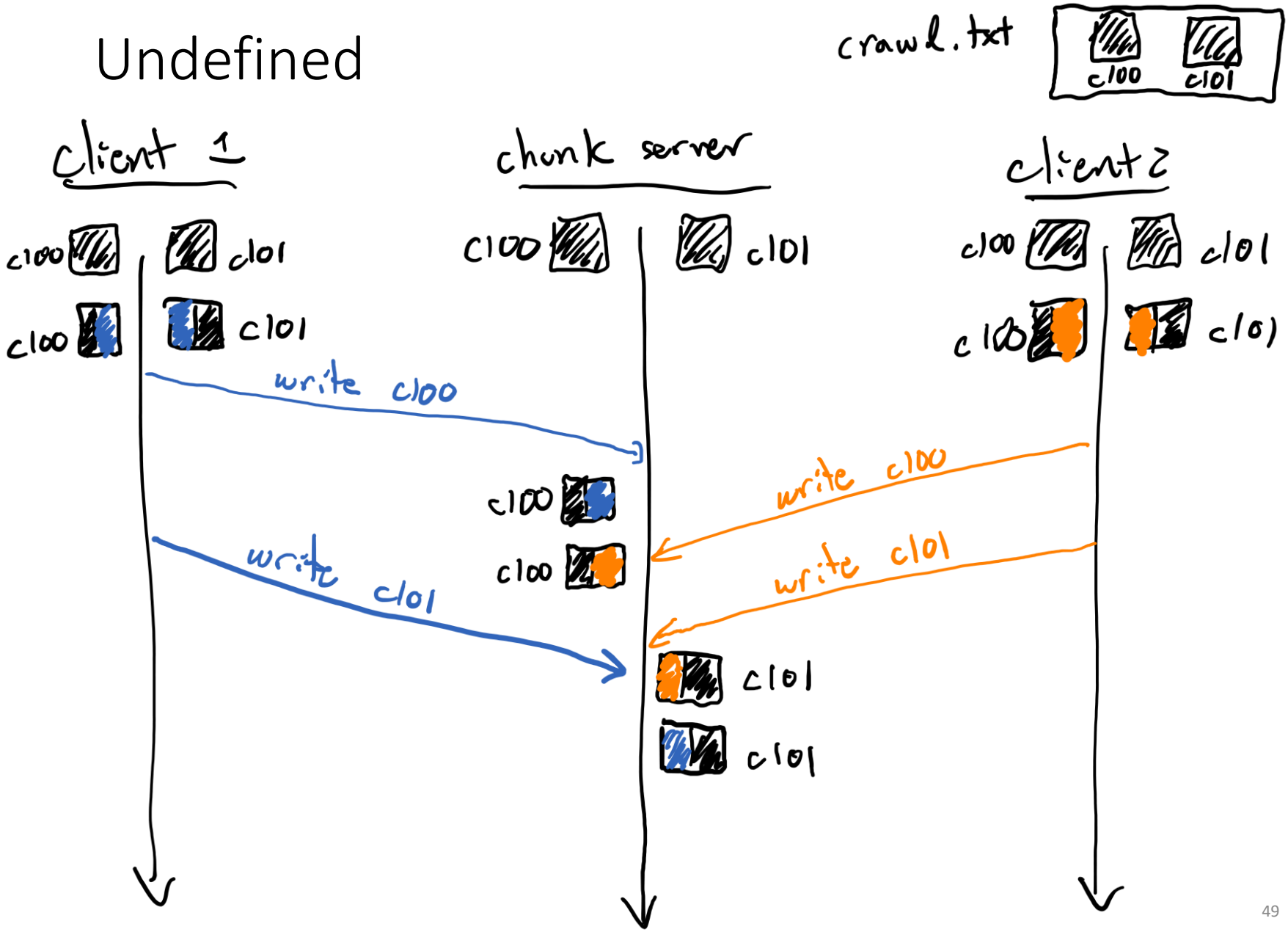
Undefined


crawl.txt
c100
c101
client 1
c100
c101
c100
c101
chunk server
c100
c101
client 2
c100
c101
c100
c101
Client 1 and client 2 modify a file across a chunk boundary

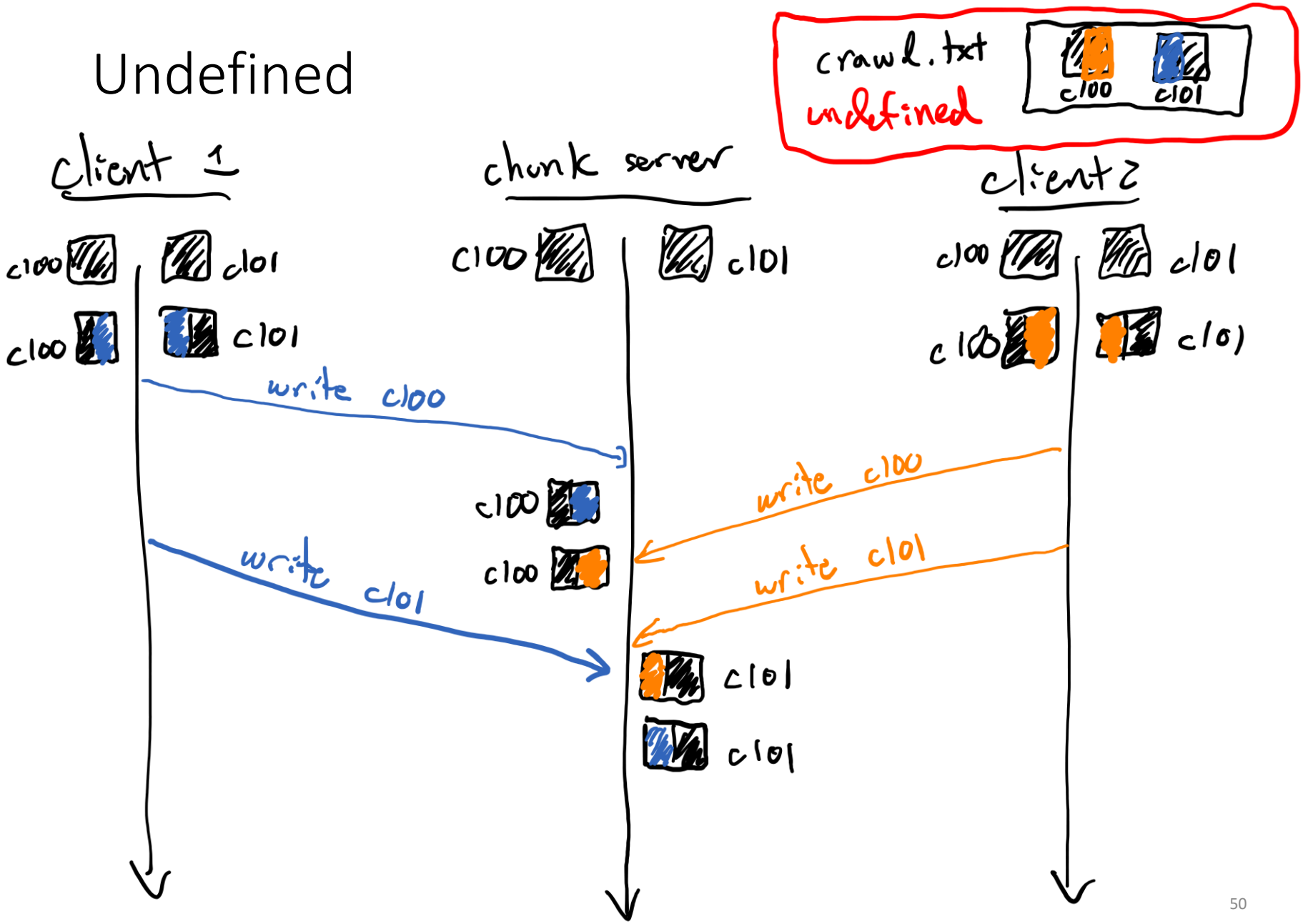
Undefined


crawl.txt
c100
c101
client 1
chunk server
client 2
c100
c101
c100
c101
c100
c101
c100
c101
c100
c101
c100
c101
write c100
write c100
c100
c100

Undefined


crawl.txt
c100
c101
client 1
chunk server
client 2
c100
c101
c100
c101
c100
c101
c100
c101
c100
c101
write c100
write c101
c100
c100
write c100
write c101
c101
c101

Undefined


crawl.txt
undefined
c100
c101
client 1
chunk server
client 2
c100
c101
c100
c101
c100
c101
c100
c101
c100
c101
write c100
write c100
write c101
c100
c100
write c101
c101
c101

Sequential consistency

- How do we fix consistent but undefined values?

- Option 1: Main server coordinates writes
  - Easy to implement
  - Slow
- Option 2: Distributed write coordination
  - Really, *really* hard to implement
  - Kinda slow

- *Sequential consistency:* Concurrent writes get the same answer as sequential writes, every time

Relaxed consistency

- How do we fix consistent but undefined values?
- Another solution: Don't bother

- *Relaxed consistency:* Sacrifice some correctness for performance

Relaxed consistency and GFS

- GFS made a deliberate design decision **not** to implement *sequential consistency*
- GFS provides *relaxed consistency*

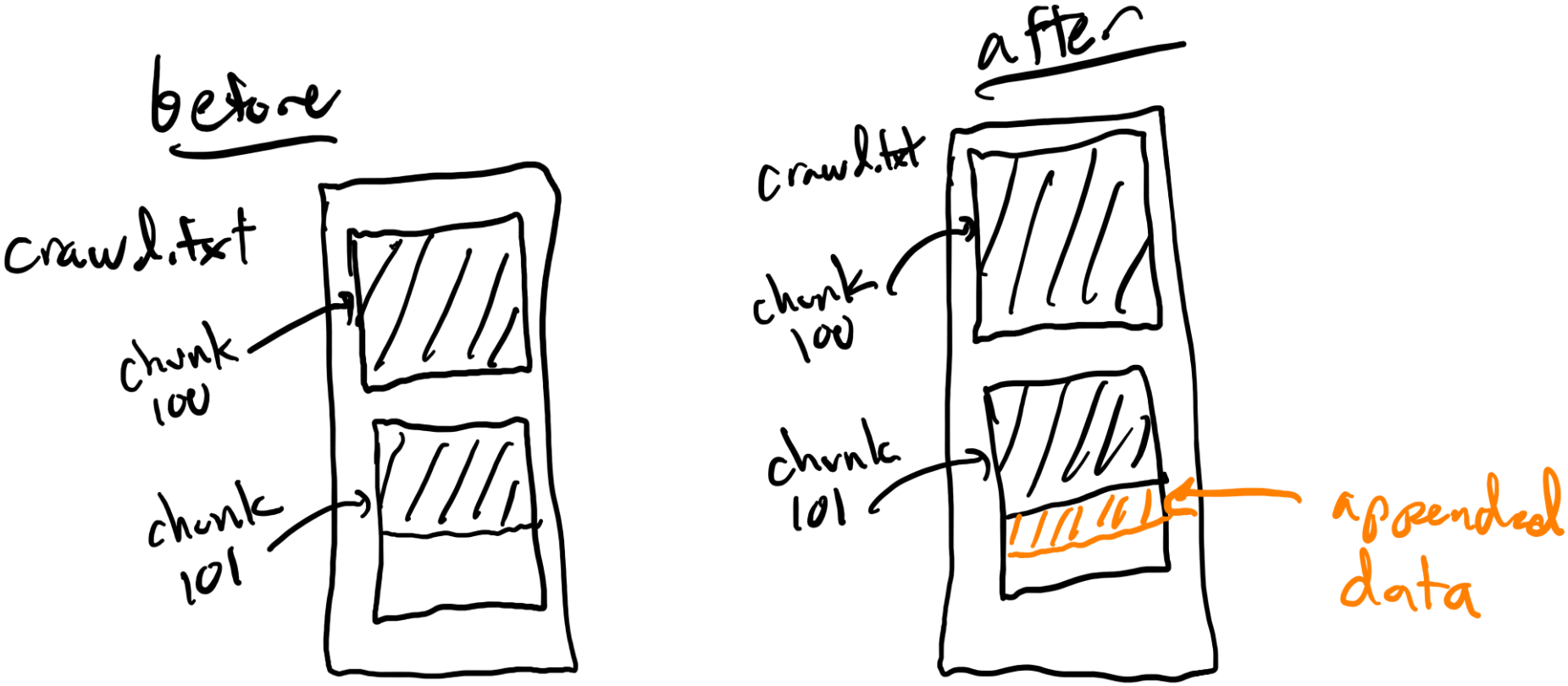
- What should the programmer using GFS do?
- Option 1: Application detects and corrects undefined file regions
- Option 2: GFS Atomic Record Append

Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- GFS design
- Read
- Write
  - Consistency
- **Append**
- Fault tolerance
- Summary

Record append

- Append: Add to the end of the file



Atomic operation

- An *atomic* operation appears to the rest of the system as if it happened instantaneously
- Avoid consistency problems

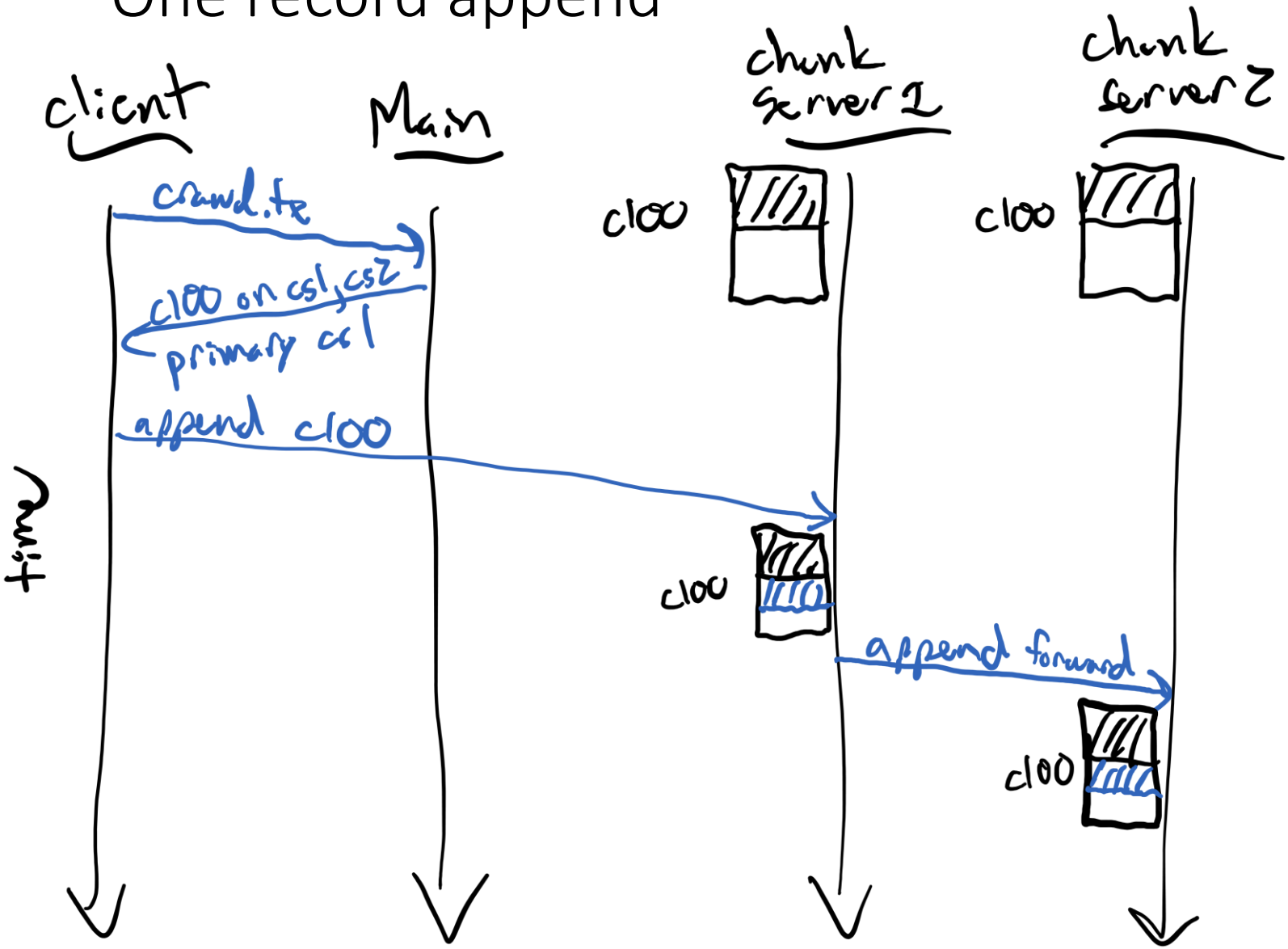
GFS atomic record append

- Insight: Record append only modifies one chunk
- Already solved the problem of consistency for one chunk
  - Write forwarding

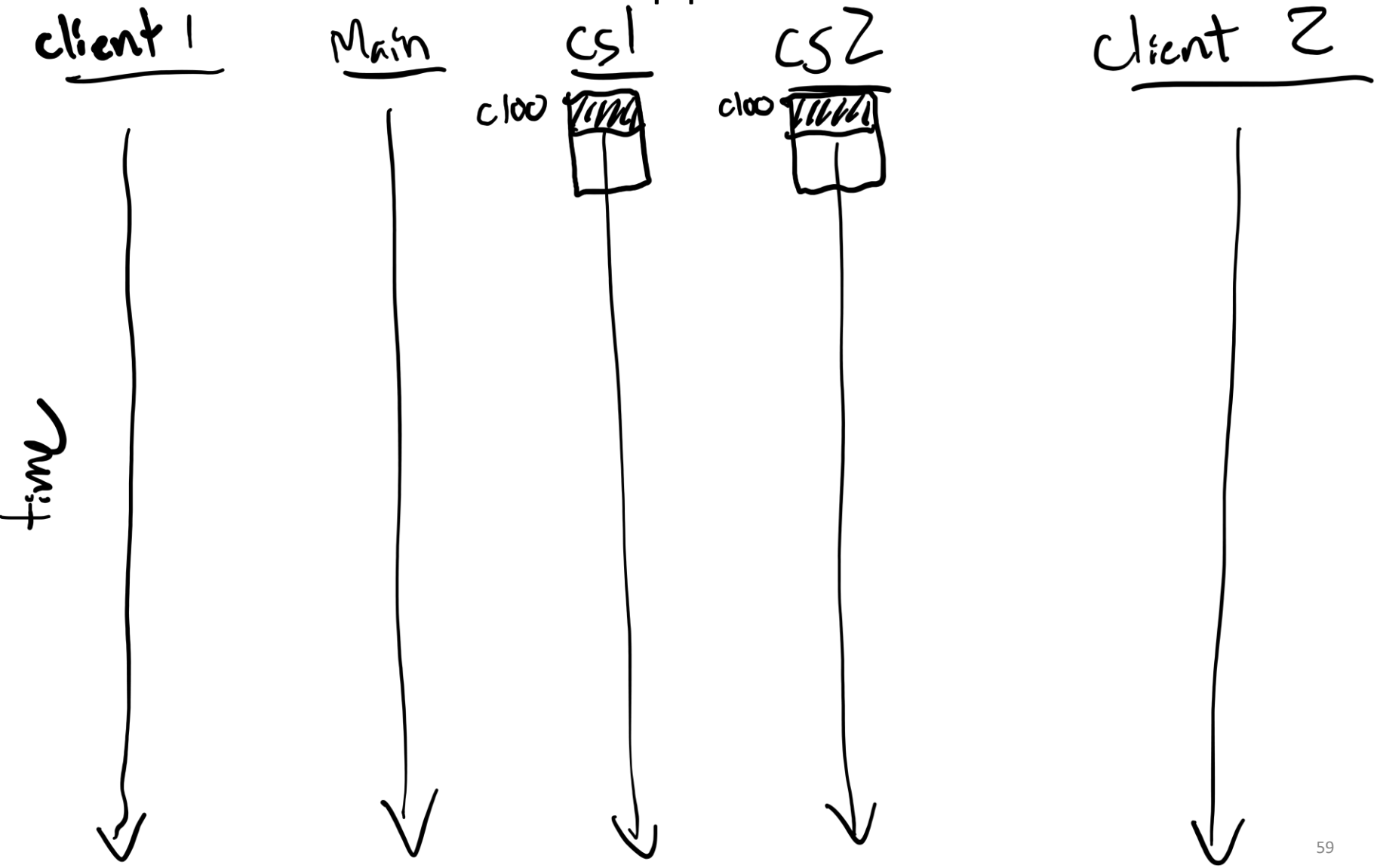
- Record append uses a similar strategy as write forwarding
- One chunkserver is designated as primary for each chunk id
- Primary forwards appends to replica chunkservers*

*This is a simplification, see the paper for full details.
http://static.googleusercontent.com/media/research.google.com/en//archive/gfs-sosp2003.pdf

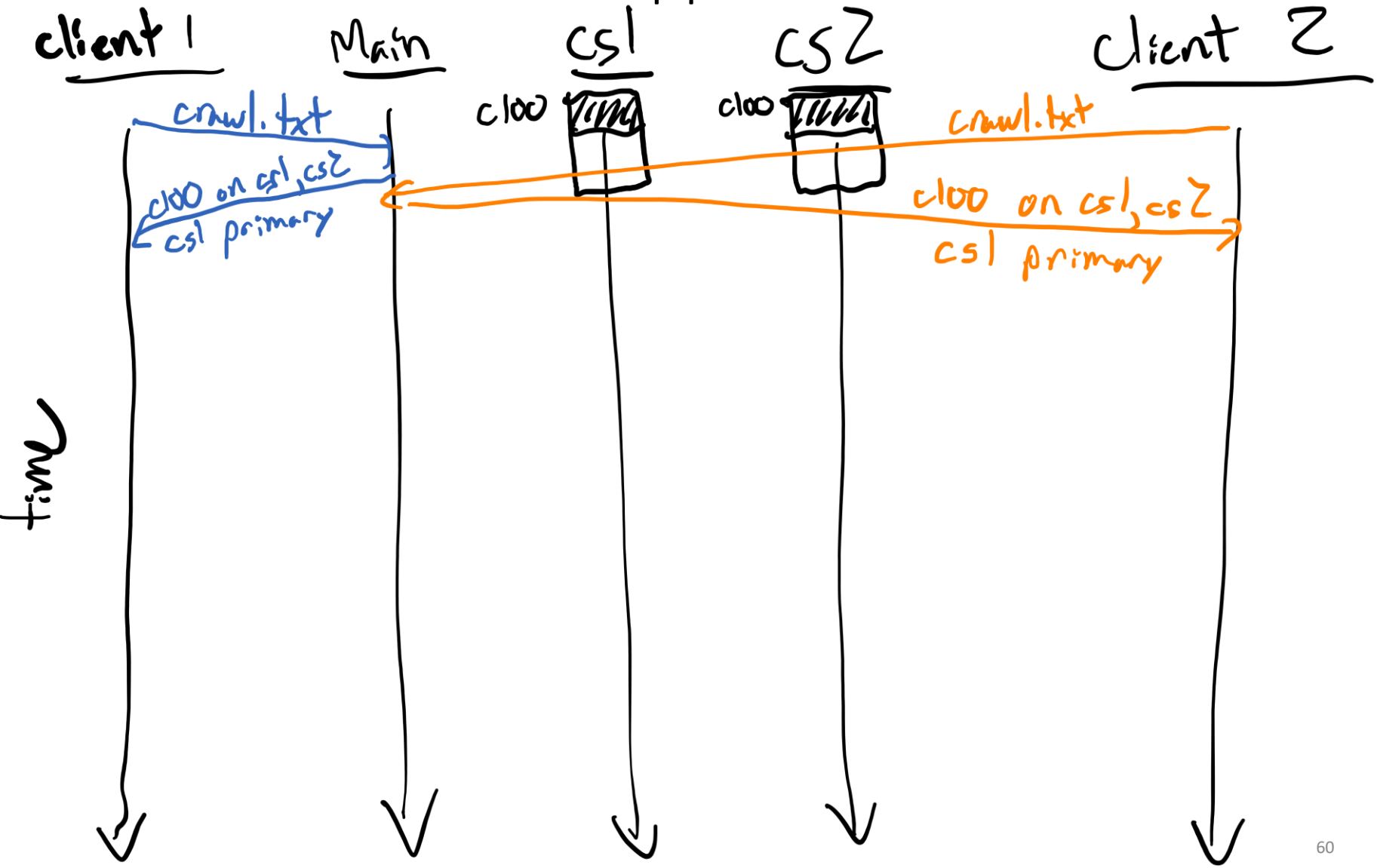
One record append



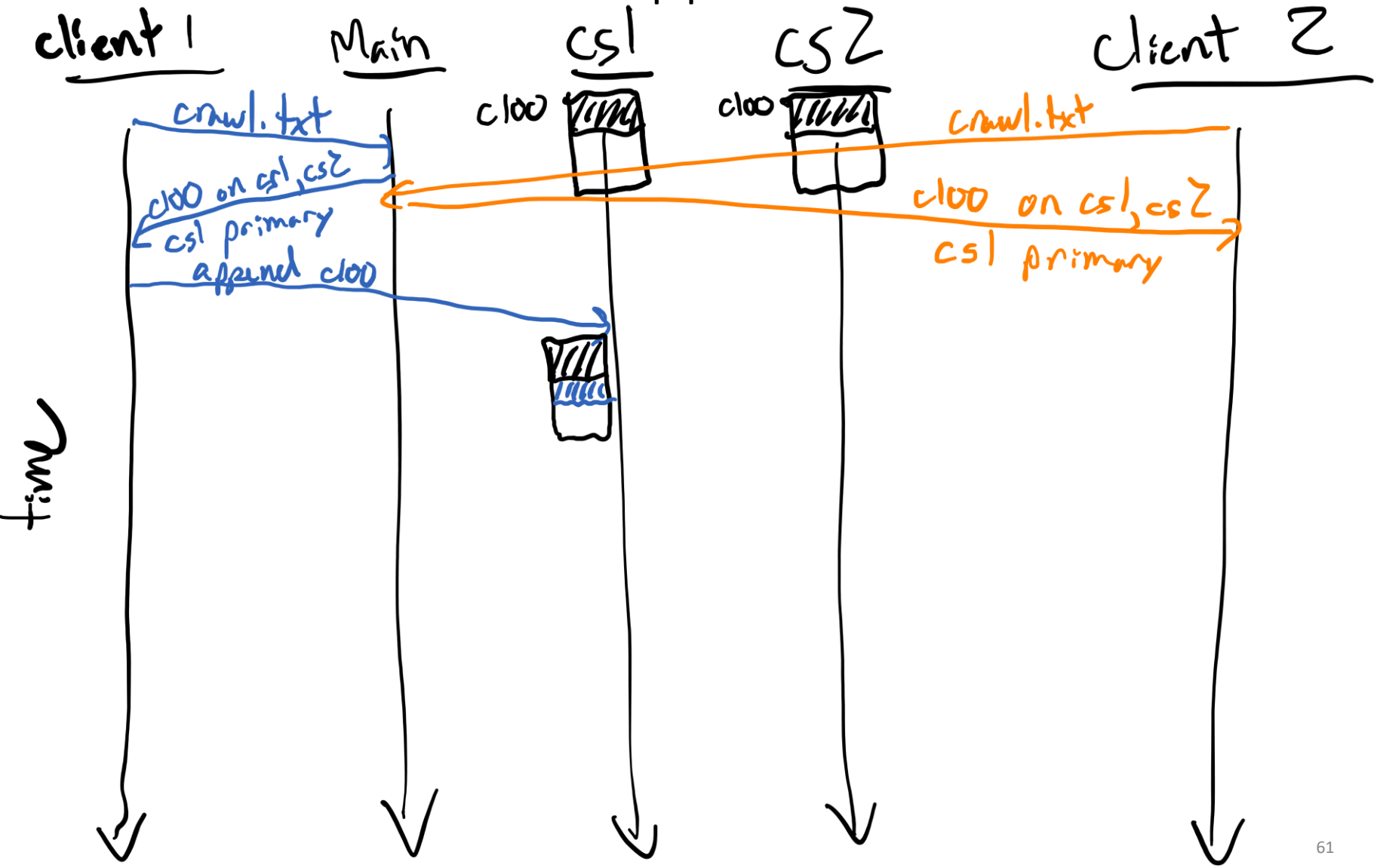
Concurrent record appends



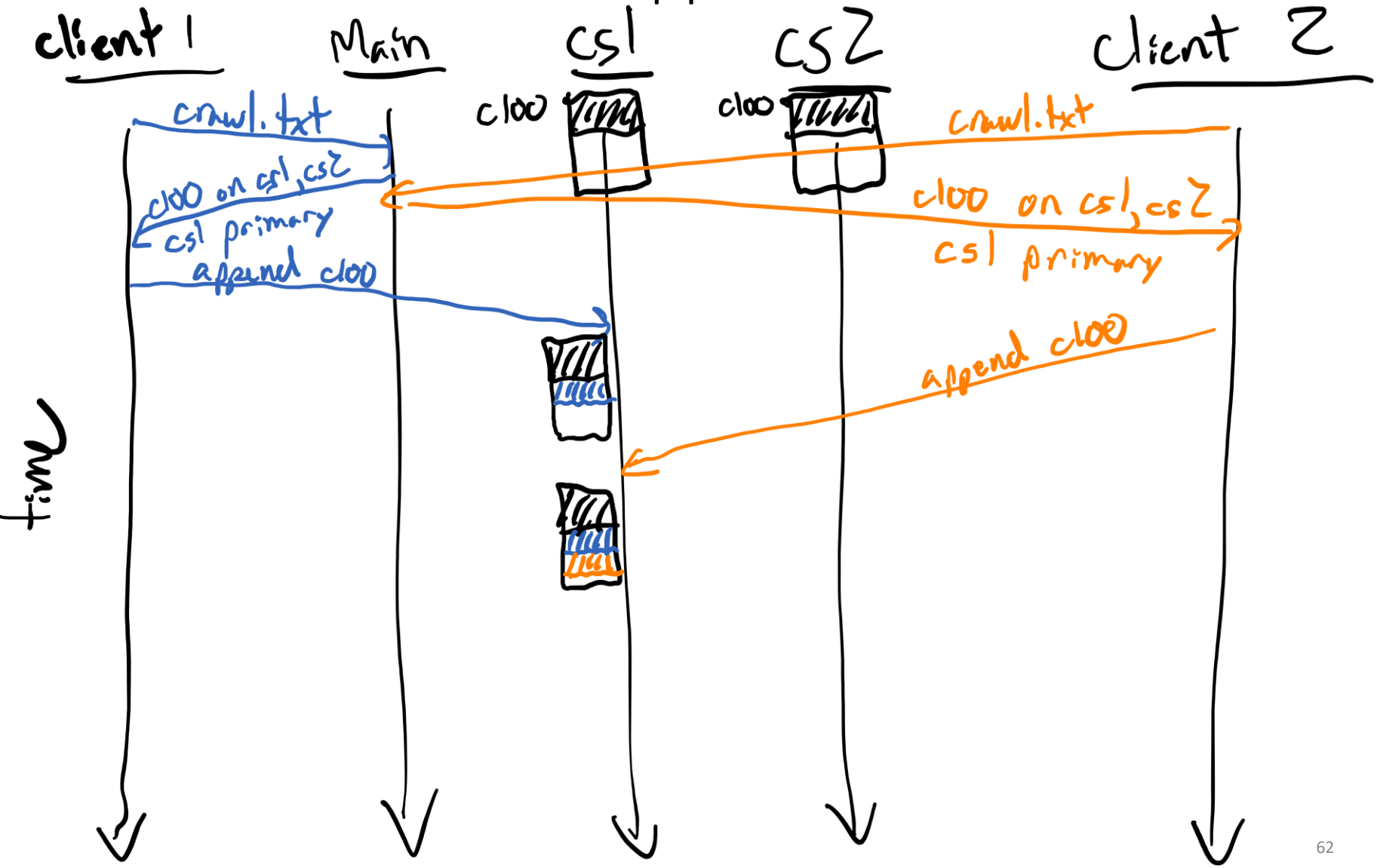
Concurrent record appends



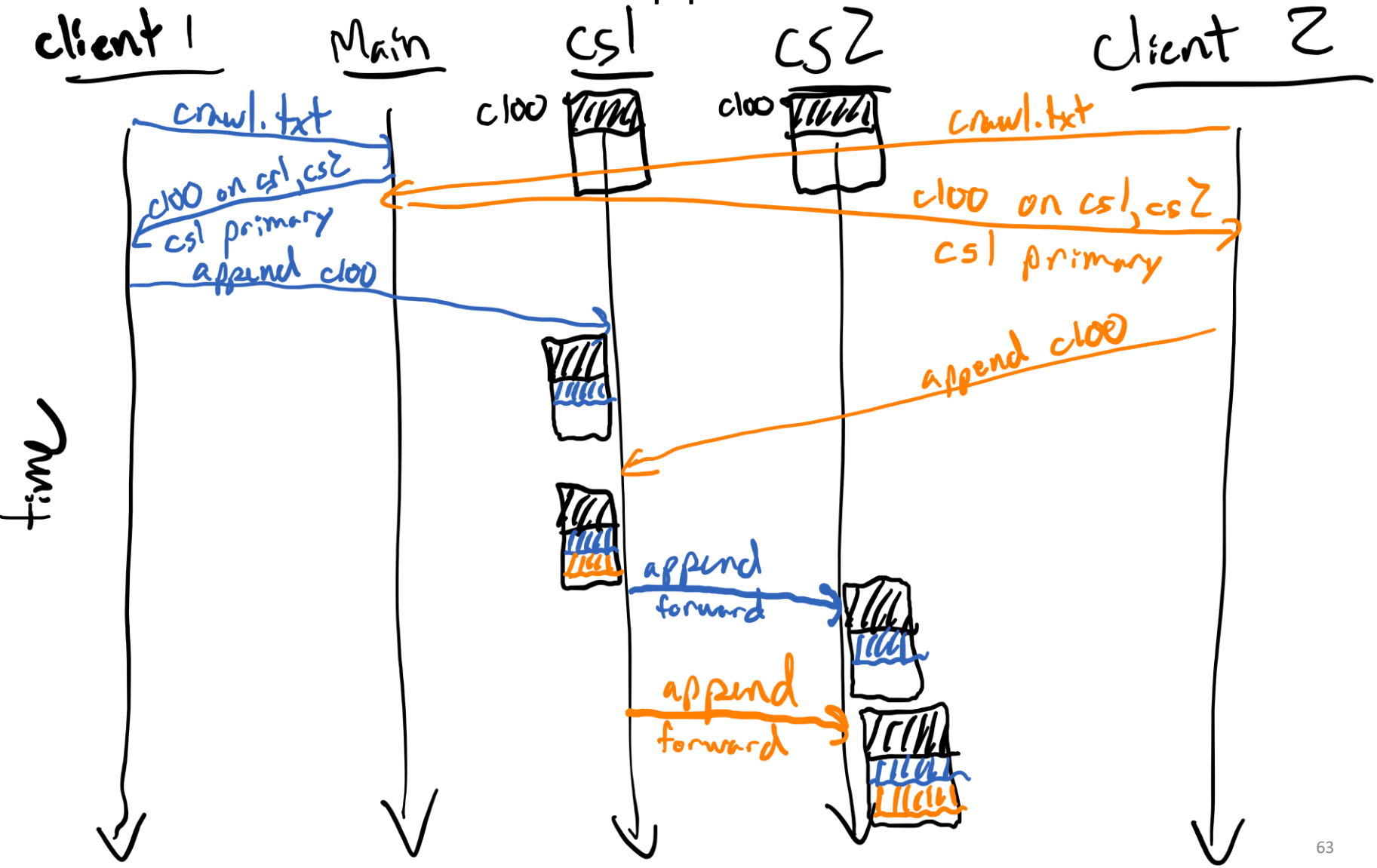
Concurrent record appends



Concurrent record appends



Concurrent record appends



GFS record append subtleties

- What if the new data is bigger than one chunk?
  - Not allowed
- What if the new data would cross a chunk boundary?
  - Pad the last chunk to "fill it up"
  - Reply to the client to try again

Optimize for append

- GFS is optimized for fast, atomic append
  - Unique to GFS

- Google applications that use GFS optimized to use append
- Example: Web crawling
  - Download a page and append it to crawl.txt

Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- GFS design
- Read
- Write
  - Consistency
- Append
- **Fault tolerance**
- Summary

Fault tolerance

- Things that can go wrong
- Chunkserver failure
- Master failure

Chunkserver failure

- Chunkservers report to master server every few seconds
  - *Heartbeat* message
- If the master loses the heartbeat, marks the server as down
- Master asks chunkservers to reduplicate data
- Typically 3 copies of every chunk

Master failure

- Master maintains critical data structures
  - filename -> chunkid map
  - chunkid -> location map
- Master writes a log to disk when data structures change

- *Shadow Main consumes log and keeps copies of these data structures up-to-date*
- *If Main goes down, Shadow Main provides read-only access until Main restarts*

Agenda

- Review: Distributed systems introduction
  - Google File System motivation
- Read
- Consistency
- Write
  - Consistency
- Append
- Fault tolerance
- **Summary**

GFS strengths

- Store lots of data
- Fault tolerant
  - Too big to back up!
- High *throughput*
  - Can read lots of data from many chunkservers at same time

GFS weaknesses

- Bad for small files
  - Chunks are ~64MB
- Master node single point of failure
- High *latency*
  - Talk to two servers to fetch any data, might need multiple chunks

Further reading

- The Google File System by Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung
  - https://static.googleusercontent.com/media/research.google.com/en//archive/gfs-sosp2003.pdf
- Storage Architecture and Challenges
  - https://cloud.google.com/files/storage_architecture_and_challenges.pdf